

我是计算机图形学

GYF

2026 年春季学期

Contents

我是计算机图形学	2
第一章 绪论	2
本节概要	2
知识点讲解	2
本节小练习	4
参考解答	4
第二章 人眼视觉系统与真实感	5
本节概要	5
知识点讲解	5
本节小练习	7
参考解答	8
第三章 计算机图形学基础	8
本节概要	8
知识点讲解	8
本节小练习	12
参考解答	12
第四章 直线、圆、椭圆生成算法	13
本节概要	13
知识点讲解	13
本节小练习	16
参考解答	16
第五章 光栅图形的扫描转换与区域填充	17
本节概要	17
知识点讲解	17
本节小练习	19
参考解答	20
第六章 裁剪、反走样与消隐算法	20
本节概要	20
知识点讲解	20
本节小练习	25
参考解答	25
第七章 真实感渲染技术	25
本节概要	25
知识点讲解	25

本节小练习	33
参考解答	33
第八章 AI 加速的实时渲染	34
本节概要	34
知识点讲解	34
本节小练习	39
参考解答	39
第九章 传统交互技术	39
本节概要	39
知识点讲解	40
本节小练习	43
参考解答	43
第十章 真实图形学应用	43
本节概要	43
知识点讲解	43
本节小练习	47
参考解答	47

我是计算机图形学

第一章 绪论

本节概要

这一章的任务是先观察图形学这门学科的全貌。读完后，你应该能回答这些基础问题：图形学到底在研究什么，图形和图像有什么区别，为什么总会提到真实性、实时性、可控性，图形系统由哪些部分组成，它是怎样一步步发展到今天的，生成式人工智能又是如何改变图形内容生产方式的。只要这一章理解了，后面学算法、建模、渲染的学习，就会感到顺很多了。

知识点讲解

初学图形学时，最容易把它理解成“画图技术”。这个理解不算错，但还不够完整。更准确地说，计算机图形学 (Computer Graphics) 研究的是：怎样用数字计算机去表示、计算、处理并显示图形。这里的“图形”既包括工程图、线框图，也包括有明暗和材质效果的真实感图像。于是你会看到，图形学并不是只有显示这一件事，它通常是一条完整链路：先建模，再计算，再输出，再交互。

接下来有一个非常关键但初学者容易忽视的问题：图形到底由什么组成？一个图形通常可以拆成两层。第一层是几何要素，比如点、线、面、体，它们决定物体的形状和位置关系。第二层是非几何要素，比如灰度、颜色、纹理、材质，它们决定“看起来是什么感觉”。可以把它理解成：几何要素回答“像什么”，非几何要素回答“看起来怎么样”。

为了把这件事做成，图形学里常用两种表示思路。第一种是**点阵法**，也就是把对象离散成像素点，直接记录每个点的属性。它的好处是显示直观、实现直接。第二种是**参数法**，用方程系数、控制点、端点、线型、颜色等参数来描述对象。它的好处是结构清楚、便于编辑、便于多视图重建。实际系统往往两者结合：内部用参数化模型管理对象，输出时再把结果栅格化到像素网格。

讲到这里，就需要把“图形”和“图像”放在一起看。**图像**通常是位图像素数据，强调最终显示结果；**图形**通常强调几何

结构和物理属性，强调可编辑与可重建。图形往往结构更强，便于修改；图像往往数据量更大，但更接近屏幕上的最终效果。图形生成不同视图能力强，但计算成本常更高；图像显示直接，但几何信息常已丢失，反向重建更困难。

有时，你还会在资料里看到 Graphics 和 Image 的工具与标准差异。图形侧常见 GKS、PHIGS、OpenGL、CGM、STEP、DXF 以及 CAD（计算机辅助设计，即利用计算机强大的计算能力和图形处理功能，辅助人们完成产品设计、工程绘图、模拟仿真等）类软件。而图像侧常见 JPEG、TIFF、JBIG 等格式和位图编辑工具。它们之间没有低级、高级之分，而是有不同的目标：一个偏结构建模，一个偏像素处理。

把二者的流程再串起来看，会更清楚。**建模** (modeling) 得到数字模型，**渲染** (rendering) 把模型变成可见图像。图像又可以进入图像处理、图像分析和识别，再反过来服务模型理解。现实世界的的数据也可以通过采集数字化进入可视化流程。科学计算可视化正是沿着这条链路发展起来的。

现在，我们可以回到图形学最核心的三个目标：**真实性、实时性、可控性**。真实性要求结果看起来可信，实时性要求系统响应足够快，可控性要求用户可以方便地改、调、交互。一个系统往往无法把三者都拉满，所以图形学的很多工程选择，本质上都是在这三者之间做平衡。

图形学的历史脉络是怎样的呢？只需抓住主线即可：早期图形系统解决“能不能显示和交互”，中期重点转向“几何建模与真实感”，现代重点是“GPU 支撑下的实时渲染与大规模图形计算”。这里折射出一条从显示、到建模、再到渲染的演进逻辑，它也决定了后续章节的知识组织方式。

如果要再把这条历史线索说得具体一点，那么有一些重要的时间节点值得我们记住。1950 年前后，MIT 的 Whirlwind I 相关显示设备标志着早期交互式图形显示的开端；随后 SAGE 系统把图形显示与实时交互进一步结合起来。1963 年，Sutherland 在 MIT 林肯实验室完成 Sketchpad，它通常被视为交互式计算机图形学的奠基性工作之一，也使“图形学之父”这个称呼和他紧密联系在一起。随后，Pierre Bezier 提出了 Bézier 曲线曲面理论，Steven Coons 提出了通过边界曲线构造曲面的思想，并较早提出 CAD 概念。这一批工作把“能显示”推进到了“能建模、能设计”。

到了 20 世纪 70 年代，光栅图形学快速发展，区域填充、裁剪、消隐等基础算法逐渐成熟，图形软件也开始走向标准化。1974 年后，SIGGRAPH 逐渐成为图形学最重要的国际学术会议之一，而 GKS、CGI、CGM、PHIGS 等标准也相继出现。对初学者来说，我们不必死记年份，但最好知道这些里程碑说明了一件事：图形学并不是单纯的算法讨论，而是在显示设备、交互方式、建模理论与软件标准共同推动下发展起来的。

图形系统组成和标准化是一个经常出现的组合。一个图形系统通常由处理硬件、图形输入输出设备、系统软件和图形软件组成。但是，逻辑输入设备与物理输入设备并非一一对应关系。逻辑输入描述的是功能抽象，物理输入描述的是硬件实现，因此同一逻辑功能可以由多种设备完成。标准化（如 GKS、CGI、CGM、PHIGS、IGES、DXF、STEP）的核心作用是统一接口和数据交换，保证跨系统协作。

如果再细分一点，图形软件还常按层次理解为基础图形软件、图形支撑软件和应用图形软件。最底层提供绘线、裁剪、填充、变换等基础能力；往上一层会把设备无关接口、图形库、驱动与交互机制组织起来；最上层才是 CAD、动画制作、科学可视化、游戏等具体应用。OpenGL 与 DirectX 就可以看作典型图形接口或开发平台的代表，它们把底层硬件能力封装为统一调用方式。像旧版 OpenGL 中的 `glLineStipple()` 用于设置虚线线型，`glLight*()` 一类函数用于设置光源参数，这类名称不一定要死记，但至少要知道它们反映的是“图形接口把常见图形操作标准化”的思想。

接下来，我们来看显示系统里的两个关键公式。首先我们先定义符号： M 和 N 分别表示屏幕水平方向、垂直方向的像素数，所以 $M \times N$ 就是一帧图像的总像素数； K 表示每个像素可取的颜色总数； $\lceil \log_2 K \rceil$ 表示描述一个像素颜色至少需要多少位； V 表示一帧图像在帧缓存中至少需要的存储位数。于是有关系式：

$$V \geq M \times N \times \lceil \log_2 K \rceil$$

这个式子表达的是**存储容量问题**。每个像素都要存自己的颜色值，因此像素总数乘以每像素位数，才是总显存需求。颜色数越多，单像素所需位数越高，所以显存压力会跟着上升。比如从 8 bit 灰度提升到 24 bit 真彩，分辨率不变时，容量需求会按位宽成倍增加。

接下来我们再看传输侧。若系统目标帧频为 F （单位通常是帧每秒），表示每秒要完整输出多少帧； T 表示显示通路每秒至少需要处理的像素量级（常称像素吞吐率或链路带宽需求）。则有：

$$T \geq M \times N \times F$$

这个式子表达的是**速度问题**，而不是“存储内容”问题。链路每秒必须把 F 帧送到屏幕，每帧有 $M \times N$ 个像素，所以先得到每秒像素总量。若把带宽定义成比特每秒，还需要再乘每像素位数（在很多教材中，我们常把该式写成像素级带宽下界，因此没有显式写颜色位宽）。

把两个式子合在一起看，我们就能直观看到工程上的约束。提高分辨率会同时推高显存和带宽压力；提高颜色精度主要增加显存压力，并在比特口径下提高链路压力；提高帧频主要增加链路实时传输压力。这也是图形系统常见的取舍来源。

最后，我们再来简单了解一下当下计算机图形学应用版图。CAD (Computer-Aided Design, 计算机辅助设计) 和 CAM (Computer-Aided Manufacturing, 计算机辅助制造)、科学可视化、数字娱乐和虚拟现实是最常见四类应用场景。我们要能说清每类场景为什么依赖图形学：CAD/CAM 依赖几何建模与精确显示，可视化依赖数据到图形的映射，娱乐依赖实时渲染与动画，VR 依赖低延迟交互与沉浸式显示。

在交互输入方面，图形系统则常采用事件方式工作，也就是由鼠标移动、按键、触笔点击等事件触发系统响应，而不是永远按固定流程机械地往下执行。这样的机制更适合图形界面和人机交互。至于 VR，除了画面分辨率、延迟和视场角外，声音也同样重要。一个合格的 VR 系统通常要求声音具备较低延迟、足够的方位感和与视觉动作一致的空间对应关系，否则即便画面正确，沉浸感也会明显下降。

本节小练习

1. 简答题

请用自己的话说明：图形与图像在“可编辑性”和“多视图生成能力”上的核心差异。

2. 判断题

逻辑输入设备与物理输入设备必须一一对应。（____）

3. 简答题

为什么图形系统设计里，总要同时考虑真实性、实时性、可控性，而不能只盯一个目标？

参考解答

1. 图形通常有几何结构和参数关系，便于编辑并生成新视图；图像主要是像素结果，显示直接但反向恢复几何结构更困难。
2. 错。逻辑输入是功能抽象，物理输入是硬件实现，一个逻辑功能可对应多个物理设备。
3. 只追求真实性会带来高计算负担，只追求实时性可能损失质量，只追求可控性会增加系统复杂度。工程上必须做平衡。

第二章 人眼视觉系统与真实感

本节概要

这一章要解决一个核心问题：为什么有些图像“看起来真”，有些“看起来假”。围绕这个问题，最需要掌握三条主线。第一条是视觉与颜色基础，解释人眼如何感知亮度、色彩与细节。第二条是显示与采样基础，解释锯齿、摩尔纹、闪烁等伪影为何产生。第三条是真实感权衡，解释为什么物理正确和视觉舒适有时不能同时拉满。读完后，你应当能够从“信号、设备、感知”三个层面分析图像质量问题。

知识点讲解

真实感并不是单一参数决定的结果，而是场景信息、显示系统、人眼感知共同作用的结果。只追求“物理数值正确”，未必视觉上舒适；只追求“观感讨好”，又可能偏离真实。完整的真实感系统通常要兼顾四件事：**几何真实性、光照与材质真实性、色彩与动态范围真实性、视觉舒适性**。

理解真实感，先从人眼视觉机制开始。人眼成像与感知链路中，角膜和晶状体负责聚焦，视网膜负责把光信号转为神经信号。视网膜中，杆状细胞主要负责亮度感知，锥状细胞主要负责颜色感知。锥状细胞在中心凹附近密度更高，所以人眼在视野中心的辨色能力更强。这个生理事实决定了图形系统不仅要“算对”，还要“看起来舒服”。另外，人眼能感知的可见光大致位于 $380 \sim 780 \text{ nm}$ 的波长范围内，颜色感知最终是对谱分布的综合响应，而不是对单一波长标签的机械识别。

色彩基础里最常考的是颜色空间。首先是 RGB。它是加色系统，显示设备最常用，颜色可写成三基色线性组合：

$$C = rR + gG + bB$$

其中， r 、 g 、 b 是三个通道系数，常见编码范围是 $[0, 1]$ 或 $[0, 255]$ 。RGB 的优点是计算直接、硬件友好，但它有色域边界，不是所有可见颜色都能由正线性组合表示。

印刷系统常用 **CMY** 或 **CMYK**。CMY 与 RGB 的归一化关系常写为：

$$C = 1 - R, M = 1 - G, Y = 1 - B$$

RGB 常把 $(0, 0, 0)$ 看成黑、 $(1, 1, 1)$ 看成白；CMY 正好相反。原因是显示器是主动发光，印刷品主要依赖反射。这个差异在判断题里经常出现。若再加入黑色通道，就得到 **CMYK**，它更适合印刷系统，因为单靠三种油墨混合得到纯黑往往不够稳定，成本与效果也不理想。

除了 RGB 和 CMY 之外，我们还常提到的是 **HSV** 颜色空间。它把颜色拆成色调 (Hue)、饱和度 (Saturation) 和明度或亮度 (Value) 三个量。HSV 的好处的优势在于更符合人对颜色选择的直觉。比如在调色软件里，我们常先选“红还是蓝”，再选“鲜不鲜艳”，最后再调“亮不亮”，这种操作逻辑就更接近 HSV 而非直接改三路 RGB 数值。

随着显示技术的发展，传统的 RGB 编码范围已难以满足高动态范围场景的需求，由此引入了 **HDR (High Dynamic Range)** 的概念。相比于传统的 SDR (标准动态范围)，HDR 不再局限于 $[0, 1]$ 的归一化亮度区间，而是通过扩展位深 (如 10-bit 或更高，也就是说能表达的颜色也更多) 和采用非线性光电转换函数，能够记录更接近真实世界的明暗细节与更宽广的色彩层次。在实时渲染中，HDR 允许像素亮度值暂时超过 1.0，这不仅解决了过亮区域“白成一片”的细

节丢失问题，也为后期通过 Tone Mapping（色调映射）将原本超高的色彩和亮度信息，映射到显示器当前能支持的最高亮度上，以模拟人眼视觉补偿，提供了更大的计算空间。

另一个高频知识点是**异谱同色**。颜色可以用谱分布描述，但谱分布与感知颜色并非一一对应，不同谱分布可能产生同样颜色感知。这也是“为什么颜色模型是感知近似，而不是光谱完整还原”的核心理由。

实际上，图像本身也可以从信号角度理解。严格地说，数字图像可以看成是一个二维离散函数 $f(x, y)$ ，定义域是一格一格排列的像素位置，而函数值就是各像素上的灰度、RGB 或 RGBA 等颜色信息。这样理解后，后面讲采样、混叠和反走样时就会自然很多，因为我们处理的本来就是一个离散采样后的二维信号。

接下来，我们来了解显示系统参数与伪影。在学习时，我们应当重点抓三个概念：分辨率、帧频、带宽。分辨率决定空间细节上限，帧频决定时间更新频率，带宽决定系统单位时间可处理的信息量。显示参数之间存在硬约束，常见写法是：

$$V \geq M \times N \times \lceil \log_2 K \rceil$$

$$T \geq M \times N \times F$$

第一式里， V 表示显存容量需求， $M \times N$ 是每帧像素数， K 是每像素可表示的颜色数。第二式里， T 表示单位时间的数据吞吐需求， F 表示帧频。这个式子，我们在第一章中就已经见到过了。可以直接看出，分辨率、颜色精度、帧频任意提高，都会推高系统负担。

从“看得舒服”角度，刷新不足会引发闪烁。常见近似关系是：

$$f_{\min} = \frac{1}{t_p}$$

其中， t_p 是荧光余辉特征时间，它指的是视觉暂留时间。当一个光刺激消失后，人眼视网膜上的感光细胞并不会立即停止放电，影像会在大脑中停留一小段时间（通常在 0.05 到 0.1 秒左右，视光线强度而定）。 $f_{\min}$ 是稳定显示所需的最低刷新频率。即为了让大脑觉得画面是“连续且稳定”的，每秒钟至少需要更新多少次。

如果两次图像刷新的间隔时间 T 大于 t_p ，在前一帧留下的“余波”消失后，下一帧还没来。这时大脑就会感知到“黑场”，也就是我们常说的闪烁（Flicker）。为了不让这种“空窗期”出现，刷新的周期必须小于或等于视觉暂留的时间。

这部分知识和显示器结构的联系也很紧密。传统 CRT 依赖电子束逐行扫描荧光屏，因此刷新频率不足时更容易出现闪烁。为了在有限带宽下缓解这个问题，曾广泛使用**隔行扫描**：一帧分成奇数场和偶数场，先扫一半行，再扫另一半行，以较低传输代价换取较高表现刷新速度。它的代价是时间分辨率和空间分辨率之间需要折中，快速运动场景中还可能引入伪影。LCD 则不依赖电子束逐点扫屏，闪烁问题通常弱一些，但它也有自己的指标，例如分辨率、响应时间和点距。所谓点距，就是相邻发光单元之间的距离，点距越小，画面通常越细腻。

本章最重要的理论就是**频率与采样**。

按照之前所学，你可能会认为图像就是 RGB 的像素阵列。但实际上这也是经过从真实世界采样而来。而即便是已经存在了电脑中的图像，也无法逃脱采样的这件事，譬如说要把一个 4000×4000 的图片二次处理为 400×400 ，那么每 10 个像素里只能挑 1 个像素显示，又要涉及采样和频率。因此，我们必须关心这个问题。

在这个问题中，我们知道，图像可看作二维信号：高频对应边缘和细节（比如亮度变化极快，像黑白相间的栅栏，它就像

去采样一段短波、高频信号)，低频对应平滑区域（如果亮度变化很慢，比如蓝天，它就像去采样一段长波、低频信号）。采样率不足时，高频会被误映射为错误低频，形成混叠，典型表现就是锯齿、摩尔纹和闪烁。

奈奎斯特条件是核心：

$$f_s \geq 2f_{\max}$$

其中， f_s 是采样频率， $f_{\max}$ 是信号最高频率。若条件不满足，就会出现频谱重叠，无法无失真重建。无论是图像缩放、纹理映射还是视频显示，这条约束都在起作用。

这涉及到数字信号处理的问题。如果对相关问题感兴趣，而又没有上过这门课程，可以前往

<https://github.com/23300240002/Homework-Archive/tree/main/Digital%20Signal%20Processing>

观看相关课程的学习资料。

针对混叠现象，计算机图形学在工程实践中演化出了**反混叠**（Anti-Aliasing）这一核心策略。目前主流的处理方法可归纳为三类：首先是超采样（SSAA），它通过在高于目标分辨率的尺度下进行渲染并对结果进行下采样（先把图像放得比原来更大，再缩得比原来更小，如此消除锯齿和增强平滑性），虽然能提供最顶级的图像质量，但因巨大的计算开销和显存占用而代价高昂；其次是多重采样（MSAA），它采取了更聪明的折中方案，仅在物体边缘进行多点采样，从而在视觉质量与处理速度之间找到了极佳的平衡点；最后是后期抗锯齿（如 FXAA、SMAA 等），这类方法直接在渲染完成后的二维图像上通过算法寻找并平滑边缘，具有开销极低、部署极其简便的优势。

此外，纹理过滤技术中的双线性、三线性以及各向异性过滤，其本质也是在纹理采样过程中，针对不同距离和视角下的频率成分进行精细的采样与重建优化，从而确保贴图细节在缩放或倾斜视角下依然能保持平滑且真实的视觉表现。

最后让我们回顾一下客观真实与主观真实之间的权衡。客观真实强调物理一致性，主观真实强调视觉接受度。二者并不总一致。比如 HDR 数据能描述更真实的亮度范围，但显示设备动态范围有限，必须通过色调映射压缩；压缩策略不同，视觉感受可能明显不同。VR 场景中也常出现类似取舍：为了降低眩晕和延迟，系统有时会优先保证高帧率与稳定交互，而不是把物理细节推到极致。

因此，本章真正要建立的是一种判断能力：当图像质量下降时，我们不只是一要考虑算法是否正确，还要同时检查采样是否足够、显示参数是否匹配、人眼感知是否被充分考虑。

本节小练习

1. 判断题

不同谱分布可能产生同一种颜色感知，这种现象叫异谱同色。（____）

2. 简答题

请解释行频、帧频、刷新频率的关系，并说明刷新不足为什么会产生闪烁感。

3. 简答题

请用一句话到两句话说明：奈奎斯特条件与锯齿、摩尔纹之间有什么关系。

参考解答

1. 对。异谱同色说明谱分布与感知颜色不是——对应。
2. 行频是每秒扫描线条数，帧频是每秒完整重绘次数，刷新频率反映重绘速度。刷新不足时，亮度在视觉暂留窗口内不能稳定融合，会表现为明暗起伏，即闪烁。
3. 当采样频率低于奈奎斯特下界时，高频信息会折叠成错误低频成分，锯齿和摩尔纹就是这种混叠在空间域的常见表现。

第三章 计算机图形学基础

本节概要

这一章会把“图形是怎么被摆放、描述、拍下来并显示出来的”这条主线完整讲清楚。你会看到三块内容：第一块是**图形变换**，解决“物体怎么移动、旋转、缩放和投影”；第二块是**图形表示**，解决“复杂形体怎么在计算机里描述”；第三块是**图形成像**，解决“三维场景如何变成二维图像，并经过相机和渲染流程得到最终结果”。学完后，你应该能够顺着一条完整流程理解从几何数据到屏幕像素的全过程。

知识点讲解

这一章信息很多，但从理解方面来看，最值得重点吃透的是三条主线：**变换与齐次坐标**、**曲线曲面表示与连续性**、**投影与成像**。

我们先看第一条主线：**变换**。图形学里，变换就是把点从一个位置映射到另一个位置。若把原点写成 p ，变换后写成 p' ，最核心关系是：

$$p' = Mp$$

其中， M 是变换矩阵。这个式子看起来简单，但它直接决定了平移、旋转、缩放、投影这些操作如何统一计算。对初学者来说，最容易踩坑的是“顺序问题”。如果先做 M_1 再做 M_2 ，总变换是：

$$p'' = M_2M_1p$$

这说明矩阵乘法顺序和操作顺序相反，也解释了为什么“先缩放后平移”和“先平移后缩放”通常结果不同。

为了让平移也能写进同一套矩阵乘法，图形学引入齐次坐标。二维点 (x, y) 常扩展为 $(x, y, 1)$ ，三维点 (x, y, z) 常扩展为 $(x, y, z, 1)$ 。为什么需要这么麻烦呢？这是因为正常情况下，平移操作只能表示为向量的加法，而线性变换（如旋转、缩放）需要表示为矩阵乘法。这会导致一个非常麻烦的工程问题：如果你想让一个物体“先旋转、再平移、再旋转、再缩放”，那么最后的计算表达式就会诸如：

$$p' = M_4(M_3(M_2(M_1p + t_1) + t_2) + t_3) + t_4$$

这种计算方法不仅冗长，而且无法简化，计算机必须一步步拆开来算，效率极低。

而如果我们引入了齐次坐标（增加了一个维度），就可以把平移也写成矩阵乘法，比如我们把 (x, y) 扩展为 $(x, y, 1)$ ，那么平移 $t = (t_x, t_y)$ 就可以表示为：

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

你看，原本属于加法的平移，现在变成了矩阵乘法的一部分。这样一来，所有的变换（旋转、缩放、平移）都可以统一成矩阵乘法，复合变换也就可以通过简单的矩阵相乘来实现了，大大简化了计算过程，提高了效率。

齐次坐标还有一个常考性质：它可以表示**无穷远点**，这和透视中的灭点概念直接相关。这又是什么意思呢？比方说在三维空间中，正常的坐标是 (x, y, z) ，我们把它扩展为齐次坐标 (x, y, z, w) 。在齐次坐标系中， (x, y, z, w) 对应到真实三维空间的坐标是：

$$(P_x, P_y, P_z) = \left(\frac{x}{w}, \frac{y}{w}, \frac{z}{w} \right)$$

在刚开始放置物体时，我们通常把 w 设置为 1，这样就能直接得到原来的坐标。但当我们后续需要拍照、投影或者进行图形变换时，这里的 w 就不一定是 1 了。比如说，在透视投影中，物体离相机越远， w 就会变得越来越大。当 w 趋近于无穷大时，物体在屏幕上的位置就会趋近于一个固定点，这个点就是我们常说的灭点。

二维与三维的基本变换类型需要掌握：平移、缩放、旋转、对称、错切（类似一个原本是长方形的物体，被从顶部往一侧推动，变成了平行四边形。此时物体形状发生了改变，但面积保持不变。此时，物体的某一个轴的坐标保持不变，而另一个轴坐标的变化取决于第一个轴坐标的值）。若按性质再分层，可以把常见变换分为刚体变换、相似变换、线性变换、仿射变换和投影变换。刚体变换保持长度和角度，相似变换额外允许均匀缩放，仿射变换保持直线性和平行性，而投影变换连平行关系也可能不再保持。更高频常见的是复合变换与绕任意轴旋转。绕任意轴旋转的标准思路是熟记基本流程：先把任意轴对齐到坐标轴，执行旋转，再逆过程恢复。若该轴不过原点，还要先平移到过原点再处理。

关于固定点的缩放矩阵也是常见考点。若要绕平面上一点 (x_f, y_f) 做缩放，本质上就是先把该点平移到原点，做缩放，再平移回去。因此：

$$M = T(x_f, y_f)S(s_x, s_y)T(-x_f, -y_f)$$

展开后可写成

$$\begin{bmatrix} s_x & 0 & x_f(1 - s_x) \\ 0 & s_y & y_f(1 - s_y) \\ 0 & 0 & 1 \end{bmatrix}$$

这个公式并不难记，关键是理解“先移到原点再移回来”的构造思想。

另一个高频点是**法向量变换**。之前提到，在变换时，顶点坐标可直接乘模型矩阵得到变换后的点坐标。然而，如果只是旋转或位移，那么法向量也可以通过乘以 M 来同步变换，但是在非均匀缩放或错切下，法向量就不能这么做变换了。

举一个简单的例子，假设有一条斜线，它的切向量是 $(1, 1)$ ，法向量是 $(-1, 1)$ 。如果我们对这个线段进行一个错切，只在 x 轴拉伸，那么切向量可能变成 $(2, 1)$ ，此时的变换矩阵是：

$$M = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$$

不难验证 M 乘以切向量 $(1, 1)$ 得到新的切向量即为预期的 $(2, 1)$ 。

然而，如果我们直接用同样的矩阵 M 来变换法向量，就会得到：

$$M \cdot \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} -2 \\ 1 \end{bmatrix}$$

此时我们再检验时就会发现新的法向量和切向量不再垂直：

$$\begin{bmatrix} 2 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} -2 \\ 1 \end{bmatrix} = -4 + 1 = -3 \neq 0$$

因此，法向量必须使用原变换矩阵的**逆转置**进行变换，才能保证法向量和切平面关系不被破坏。这个逆转置的意义实际就是抵消了非均匀缩放或错切带来的变形，确保法向量在变换后仍然正确地表示表面方向。

标准结论是：

$$n_{WS} = (M^{-1})^T n_{OS}$$

这里， n_{OS} 表示对象空间的法向量， n_{WS} 表示世界空间的法向量（世界就是指变换后的空间）， M 是模型变换矩阵。

接下来，我们再来看第二条主线：**图形表示**。也就是说要搞清楚如何在计算机中把图形表示出来。三维形体有两类常见的描述方式：**数据模型与过程模型**。

其中，数据模型直接存顶点、边、面和拓扑来表示模型，代表方法包括 B-Rep（边界表示法，通过描述物体的点、边、面来定义之）和 CSG（空间实体几何，通过基本几何体（球、方块、圆柱）的布尔运算（并、交、差）来构造物体）。而过程模型更强调“按规则生成”，适合参数驱动建模。

比如说我们需要描述一棵树，那么数据模型会存几万个树叶的坐标；然而，过程模型只存一条规则：树干长到 10cm 就分叉，分叉角度 30 度，重复 5 次。

曲线部分是这章最容易和考题直接对应的内容。在这里，我们需要研究的问题是：如何用数学方法描述曲线？以及，两个曲线段连接时，什么样的连接才算“平滑”？

在这里，连续性符号是核心考点： C^0 表示位置连续（两个曲线段连接在一起，没有断点即可，但可能存在尖角）； G^1 表示切线方向连续（方向也相同，但大小可以不同，即导数值不必相等）； C^1 表示切向量连续（一阶导数要求相等）； C^2 则要求二阶导也要连续。很多题目本质就是在问两个曲线段连接是否平滑到某一阶。

此外，还有两种常见的曲线表示方法：**Bézier 曲线**和**B 样条曲线**。Bézier 曲线由一组控制点定义，具有全局影响（调整一个控制点会影响整条曲线），适合简单形状；B 样条曲线则由多个段组成，每段由局部控制点定义，具有局部可控性（调整一个控制点只影响附近的曲线段），适合复杂形状。要掌握 Bézier 与 B 样条的关系：Bézier 直观、全局影响明显；B 样条局部可控、拼接更灵活。

值得注意的是，Bézier 曲线有端点性质，也就是曲线经过首末控制点；它还有对称性，若把全部控制点顺序反过来，曲线几何形状不变，只是参数方向相反。而 B 样条则更强调局部性与凸包性质：某一段曲线只受附近少数控制点影响，而且这段曲线位于相关控制点形成的局部凸包内。正因为局部性强，B 样条才特别适合交互式建模。

在曲线描述问题中，**节点插入**与**NURBS**这两种方法也值得关注。节点插入的技术目标，不是为了改曲线形状，而是为了在不改变原曲线几何形状的前提下增加控制自由度，便于后续分段处理、局部编辑和拼接。NURBS (Non-Uniform Rational B-Splines, 非均匀有理 B 样条) 则是在 B 样条基础上进一步加入非均匀节点与权因子。它的重要性在于既能表示自由曲线曲面，又能统一表示圆、圆锥曲线等标准解析形状，因此在 CAD/CAGD 中地位很高。当然，权因子也不是越大越好，选取得不合适时反而可能引起形状畸变。

刚才讨论了曲线，对于曲面部分，我们需要掌握三个关键词：**参数曲面**、**张量积**、**生成方式**。

参数曲面常写成 $P(u, v)$ 。这是为什么呢？在数分中我们学过，曲线是一个参数函数 $P(t)$ ，这里的 t 实际上就是一个参数，表示曲线上的具体位置。然而，因为曲面是二维的，我们需要两个参数 u 和 v 来描述上面某个点的位置。比如说一个球面上的某个点就可以用 $P(u, v) = (r \sin u \cos v, r \sin u \sin v, r \cos u)$ 来表示，其中 u 是从北极到南极的纬度参数， v 是绕球体旋转的经度参数。

张量积实际就是矩阵式的网格结构。比如说我们有两条曲线 $C_1(u)$ 和 $C_2(v)$ ，它们分别定义了两个方向上的曲线，那么我们就可以通过张量积的方式来生成一个曲面：

$$P(u, v) = \sum_{i=0}^n \sum_{j=0}^m B_i(u) B_j(v) P_{ij}$$

这里的 $B_i(u)$ 和 $B_j(v)$ 是两个方向上的基函数， P_{ij} 是控制点。通过调整这些控制点，我们就可以改变曲面的形状。因而，一条曲线 $C(u)$ 就可以表示为 $C(u) = \sum_{i=0}^n B_i(u) P_i$ ，而一个曲面 $P(u, v)$ 就可以表示为上述的双重求和形式。

我们知道，Bézier 曲线的阶次等于控制点数减一，因此三次 Bézier 曲线需要 4 个控制点。那么，如何得到一个双三次 Bézier 曲面呢？答案是需要 4×4 个控制点。如果我们把 4 条这样的曲线并排摆放，再用另外 4 条曲线横向穿过它们，就像织布一样，形成一个 4×4 的控制点网格，这样就可以定义一个双三次 Bézier 曲面了。

曲面还可以通过旋转或扫掠来生成。旋转曲面是轮廓线绕轴旋转，广义圆柱是轮廓沿路径扫掠，这两个概念在选择题和名词解释里都较常见。

最后，我们再来看第三条主线：**投影与成像**。三维到二维最核心的区分就是**正交投影**和**透视投影**。正交投影保持平行关系，工程制图常用；透视投影则包含近大远小，更接近真实视觉。

例如，若投影平面为 $z = d$ ，当我们进行正交投影时，直接将 z 坐标丢弃，而 x 和 y 坐标不变即可。然而如果我们进行透视投影，那么透视式应当写为：

$$T(x, y, z) = \left(\frac{d}{z}x, \frac{d}{z}y, d \right)$$

这里的 d 是投影平面与相机的距离，也就是相机的焦距。这个式子要理解两个点：一是深度 z 越大，投影尺度越小；二是灭点现象来自除法项。

关于透视投影，还可以顺手记一个判断题常见结论：三维空间里主灭点的个数最多是 3 个，因为最多只有三组互相独立、且不平行于投影平面的坐标轴方向平行线，会分别汇聚到不同的主灭点。

在二维显示流程里，还常出现**窗口到视图区变换**。窗口是用户坐标系中真正关心的那一块区域，视图区则是设备屏幕上实际拿来显示的区域。若窗口边界为 WXL, WXR, WYB, WYT ，视图区边界为 VXL, VXR, VYB, VYT ，则窗口中点 (X_w, Y_w) 映射到视图区点 (X_s, Y_s) 的常见公式为：

$$X_s = VXL + \frac{X_w - WXL}{WXR - WXL}(VXR - VXL)$$

$$Y_s = VYB + \frac{Y_w - WYB}{WYT - WYB}(VYT - VYB)$$

这组公式本质上就是分别对 x 、 y 两个方向做线性缩放和平移。若窗口和视图区长宽比例不一致，显示图形就会发生拉伸变形。

对于成像，我们不必追求底层硬件细节，但是核心的流程一定要了解。可以概括为：场景几何先经相机与投影变换，再经裁剪、光栅化、着色得到像素；而拍照链路则是镜头进光、传感器采样、ISP 处理（将传感器（Sensor）输出的原始数字信号转换为可视彩色图像的核心流程）再输出图像。这里还常配合 **FOV**（Field of View，视场角）一起理解：视场角越大，能拍到的范围越广，但透视畸变也会更明显。曝光关系常写为：

$$H = T \times E$$

其中， H 是曝光量， T 是曝光时间， E 是辐照度。它和光圈、快门、ISO 一起构成曝光控制的基本框架。

本节小练习

1. 简答题

为什么“先缩放再平移”和“先平移再缩放”通常不会得到同样结果？请用一句到两句说明原因。

2. 判断题

在错切和非均匀缩放下，法向量可以和顶点使用完全相同的变换矩阵。（____）

3. 简答题

请简要说明正交投影和透视投影在视觉效果上的核心差异，以及它们各自更常见的使用场景。

参考解答

1. 因为复合变换对应矩阵乘法，而矩阵乘法一般不满足交换律，所以顺序不同结果通常不同。

2. 错。此时法向量应使用原变换矩阵的逆转置进行变换。

3. 正交投影深度感弱但比例关系稳定，常用于工程制图；透视投影有近大远小和灭点，更接近人眼视觉，常用于真实感显示。

第四章 直线、圆、椭圆生成算法

本节概要

这一章进入图形学里十分关键的一部分：把连续几何对象变成屏幕像素。你会看到三件核心事情：第一是直线如何画得又快又准，第二是圆弧如何利用对称性减少计算，第三是椭圆为什么要分两段处理。学完后，你应当能看懂并解释 DDA、中点法、Bresenham 这几类经典算法的判别式、递推关系和初值来源。

知识点讲解

先把本章最核心的背景说清楚。计算机图形学中，我们要完成在电脑屏幕上绘制几何对象的任务。比方说我们想要展示一条直线或者一个圆，但是屏幕是像素网格，我们想展示的几何对象却是连续的。这就需要我们点亮那些“最接近”我们想要展示的几何对象的像素点，让展示出来的图像看起来尽可能接近我们想要的连续图形。

所谓**扫描转换**（也常叫**光栅化**，Rasterization），本质上是在网格里挑一组“最接近目标图形”的像素并点亮。这个过程永远伴随一个问题：连续量到离散量会产生误差，所以算法设计的关键不是追求绝对精确，而是如何在误差可接受的前提下做到高效、稳定、易实现。

在工程流程里，扫描转换通常和裁剪一起出现。更常见的顺序是先裁剪再扫描转换，因为先把窗口外不可见部分去掉，可以节省后续像素计算量。这个顺序是很重要的。

在本章中，我们主要关注的是一些基本图元的扫描转换算法，例如直线、圆、椭圆。它们是构建更复杂图形的基础。在后续章节，我们再了解如何进行更复杂的扫描转换：把这些基本图元组合成多边形、曲面等更复杂的对象。

接下来先看**直线**。第一种方法是 **DDA (Digital Differential Analyzer, 数值微分法)**。它的想法很直接：若直线方程是 $y = kx + b$ ，那么当斜率满足 $|k| \leq 1$ 时，让 x 每次加 1， y 就按斜率递推： $y_{i+1} = y_i + k$ ，然后再把 y 四舍五入成像素坐标。

需要注意的是，若 $|k| > 1$ ，就要交换步进主轴，改为沿 y 方向递推。这是由于如果斜率过大，沿 x 方向每次加 1 会导致 y 的变化过大，很多格子被跳过，最后实际绘制线条时就会出现很多锯齿和不连续的情况。通过交换主轴，我们可以保证每次递推都能覆盖所有需要点亮的像素，从而得到更平滑的线条。

DDA 的方法很好理解，但它每一步都要做浮点运算和取整，硬件实现效率不理想，所以它更像一个“教学入门算法”。除此之外，**角度 DDA 法**也可以用来生成圆弧。它不再直接沿 x 或 y 递推，而是把圆周运动看成角度逐步增加时的坐标更新。其本质仍然是“用增量替代重算”，只是递推对象从直线斜率换成了角度参数。

第二种方法是**中点画线法**。这是本章高频考点之一。它不再直接算几何距离，而是构造判别函数，用符号判断哪个候选像素更近。设当前像素是 $P(x_p, y_p)$ ，我们知道，由于 $0 < k < 1$ ，下一个候选像素就只能是右方点和右上点了。那么，我们就将它们的中点记为 $M(x_p + 1, y_p + 0.5)$ ，此时，只要判断 M 到上下两点的距离哪个更近，就能知道下一个像素应该点亮哪个了。那么，如何进行快速的判断呢？这就引入了判别函数。我们把直线写成隐式形式：

$$F(x, y) = ax + by + c$$

很显然，其中 $a = y_0 - y_1$ ， $b = x_1 - x_0$ ， $c = x_0 y_1 - x_1 y_0$ ，这些都可以预先算得。在高中的解析几何学习中我们就学过， $F(x, y)$ 的符号可以告诉我们点 (x, y) 在直线的哪一侧：如果 $F(x, y) > 0$ ，中点就在直线的上方；如果 $F(x, y) < 0$ ，中点就在直线的下方；如果 $F(x, y) = 0$ ，点就在直线上。因此，我们将判别量定义为

$$d = F(x_p + 1, y_p + 0.5)$$

若 $d < 0$ ，显然应该取右上像素；若 $d \geq 0$ ，显然就要取右方像素。

然而，即便如此，每步都要重新计算 d 的值，效率仍然不高。于是我们就引入了**增量递推**的思想。由于每次移动一个像素，我们可以预先计算出 d 的增量，这样在每一步只需要在上一步的基础上加上这个增量就可以了，而不需要重新计算整个 d 的值。

我们进行分析：继续往后一步时，显然判别式只会出现两种增量： $d \leftarrow d + a$ （往右）或 $d \leftarrow d + a + b$ （右上）。如此，每次只需要在上一步的基础上加上相应的增量，计算 d 的速度就快多了。而初值就来自起点代入中点： $d_0 = a + 0.5b$ 。

在这个过程中，因为只看符号，常把它整体乘 2 变成整数运算。你会发现，中点法的思想非常统一：先建立判别式，再把每步重算改为增量递推即可。

第三种是 **Bresenham 画线算法**。它和中点法的方向一致，但在效率上更工程化。Bresenham 通过误差项递推，把浮点运算进一步压缩到加减和比较，非常适合硬件或底层图形库实现。比方说，我们把判别式的增量改写成一个误差项 e ，每次根据 e 的符号来决定下一个像素。而这里的 k 等于 $\frac{\Delta y}{\Delta x}$ ，我们可以把它乘以 Δx 来避免浮点运算。每次横走一步， e 就加上 Δy ；如果 e 超过了 Δx ，说明需要往上走一步，此时 e 就减去 Δx 。这样就完全避免了浮点运算，效率大大提升。

由此可见，如果问“为什么 Bresenham 更高效？”，那么核心回答就是：它用整数递推替代了大量浮点和舍入操作。

直线讲完后，我们进入本章的第二个重点：**圆**。圆的起点是方程 $x^2 + y^2 = R^2$ 。

这里，如果硬算函数值再取整，计算量会偏大。我们要学习的重点还是**中点画圆法**。我们知道，圆是高度对称的对象，八分之一圆就能通过对称性复制得到整圆。比方说如果我们从圆的顶部出发，沿着第一象限的弧段往右下走 45° ，那么我们就可以通过对称性把这个八分之一圆复制到其他七个象限，得到完整的圆。因为，对于同象限的那个部分，我们把 x 和 y 互换即可；剩余的其他象限的六个部分，只需要进行对称、再交换等操作就可以了。

因此，我们不妨只考虑圆的顶部出发，沿着第一象限的弧段往右下走 45° 的那个八分之一弧段。这也是工业中常用的方法。

设判别函数

$$F(x, y) = x^2 + y^2 - R^2$$

类似前面的中点画线法：当前点为 $P(x_p, y_p)$ 时，显然候选点是右方和右下方，而中点即为 $M(x_p + 1, y_p - 0.5)$ 。因此，判别式 $d = F(x_p + 1, y_p - 0.5)$ 。

若 $d < 0$ 取右方；若 $d \geq 0$ 取右下。在这里，我们也要进行递推增量，是本章最常考的公式之一。由于到了圆上，增量不再单纯是常量 a 或 $a + b$ 了，而是要根据当前点位置来判断。具体来说，如果取右方像素，那么增量就是

$$\Delta d = 2x_p + 3$$

而如果取右下像素，那么增量就是

$$\Delta d = 2(x_p - y_p) + 5$$

大家可以自行计算，来验证上述结果的正确性。当然，这个结论也十分值得我们直接记住。

对于初值，我们取起点 $P(0, R)$ 代入中点 $M(1, R - 0.5)$ 得到：

$$d_0 = 1.25 - R$$

此外，还有 **Bresenham 画圆算法**与生成圆弧的**正负法**。这里的 Bresenham 画圆算法和前面很类似，也是对中点画圆法的进一步优化，通过整数递推来避免浮点运算。而生成圆弧的正负法则比中点法更加直接易懂，它直接通过 $F(x, y)$ 的符号来判断当前像素点是否在圆的内部、外部或者正好在圆上，从而决定下一个像素点的选择，简单来说就是根据“当前位置”的反馈立刻纠偏，因而轨迹可能会比中点法稍微更“抖”一点。在学习时，我们可以把这些方法（包括之前的中点法）都定位成同一思路的不同落地：因为它们都是先构造判别式从而避免复杂的除法、开方运算，然后利用递推来提升效率，最后通过圆的八对称性来减少计算量。

圆弧之外，我们还需要简单了解一下内接正多边形逼近与等面积正多边形的逼近。这部分通常不是计算的重点。如何简单理解呢？我们知道，内接多边形边数足够多时可逼近圆，比如说某条边的起点是 $P(x_i, y_i)$ ，终点是 $P(x_{i+1}, y_{i+1})$ ，那么就可以利用旋转递推来持续获得后续的顶点坐标（例如 $P(x_{i+2}, y_{i+2})$ 等），因而画出后续的边：

$$\begin{bmatrix} x_{i+1} \\ y_{i+1} \end{bmatrix} = \begin{bmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{bmatrix} \begin{bmatrix} x_i \\ y_i \end{bmatrix}$$

其中 α 是相邻顶点对应的圆心角。

等面积逼近，则是在面积意义上修正内接或外切偏差。因为在前面的内接多边形逼近中，虽然边数可以无限增加，但由于每条边都是内接的，所以整个多边形的面积始终小于圆的面积。我们可以调整多边形的半径，使得其比内接的稍大一点，形成的多边形轮廓部分在圆的外部，部分在圆的内部，并使得多边形的面积和圆的面积相等，从而达到等面积逼近（视觉上更加贴合）的效果。

最后我们来看**椭圆**。椭圆方程为

$$F(x, y) = b^2x^2 + a^2y^2 - a^2b^2 = 0$$

这里 a 和 b 分别是半轴长度。椭圆和圆最大的实现差异是：它虽然有对称性，但显然不能再使用八分之一对称了，最多只能使用四分之一对称。而且第一象限弧段的“前进方向”会变化，前面 x 为主导，后面要变成 y 为主导，因此算法上通常分为上半段和下半段。分段本质来自切线斜率变化，也可理解为法向量两个分量主次互换。

1. 上半段使用判别式

$$d_1 = F(x_p + 1, y_p - 0.5)$$

若 $d_1 < 0$ ，取正右像素；若 $d_1 \geq 0$ ，取右下像素。

不难推导得到：如果取正右像素，那么增量为 $\Delta d_1 = b^2(2x_p + 3)$ ；如果取右下像素，那么增量为 $\Delta d_1 = b^2(2x_p + 3) + a^2(-2y_p + 2)$ 。

上半段初值常写为 $d_{10} = F(1, b - 0.5) = b^2 + a^2(-b + 0.25)$ 。

2. 当进入下半段后，候选点改成正下和右下，需要重置判别式：

$$d_2 = F(x_p + 0.5, y_p - 1)$$

若 $d_2 < 0$ ，取右下像素；若 $d_2 \geq 0$ ，取正下像素。

如果取右下像素，那么增量为 $\Delta d_2 = b^2(2x_p + 2) + a^2(-2y_p + 3)$ ；如果取正下像素，那么增量为 $\Delta d_2 = a^2(-2y_p + 3)$ 。

下半段的初值则需要从上半段走完后的最后一个像素点开始计算，为 $(x_{last} + 0.5, y_{last} - 1)$ 。

这也是为什么椭圆中点法比圆中点法更复杂一些，因为它天然就需要两阶段的判别。

回顾本章，我们就能发现：直线、圆、椭圆看似对象不同，但算法骨架高度一致，都是在候选像素选择上构造判别式，并把判断过程写成高效递推。掌握这个共性，后面学扫描线填充、裁剪和反走样，我们就会轻松很多。

本节小练习

1. 判断题

在 $|k| \leq 1$ 时，DDA 画线法可以让 x 每步加 1，并用 $y_{i+1} = y_i + k$ 递推。（____）

2. 简答题

中点画圆法里，为什么只计算一个八分圆就可以得到整圆？

3. 简答题

椭圆中点算法为什么要分上半段和下半段两段来处理？

参考解答

1. 对。在该斜率范围内，沿 x 方向步进可稳定覆盖像素，配合取整即可得到直线近似。
 2. 因为圆关于 x 轴、 y 轴和直线 $y = x$ 等具有对称性，算出一个八分圆后可通过对称映射得到其余部分。
 3. 因为椭圆弧在第一象限内的几何变化速度会发生主次切换，上半段与下半段的候选像素关系不同，判别式和递推更新也需要分开处理。
-

第五章 光栅图形的扫描转换与区域填充

本节概要

这一章的核心任务是解决一个非常实际的问题：我们已经有了多边形边界，接下来怎样把它稳定、快速地填成一片像素区域。你会看到两条主线。第一条是“多边形扫描转换”，它从顶点和边界出发，算出内部像素。第二条是“区域填充”，它从一个种子点出发，把颜色扩展到整个连通区域。学完后，你应该能说清楚逐点判断法、扫描线算法、边界标志法、种子填充法各自的思想、优缺点和典型适用场景。

知识点讲解

先从最基础的问题开始。我们为什么要专门学“填充”？因为在屏幕里，图形最终一定是像素集合。一个多边形如果只画边，不填内部，它就还只是轮廓。要做真实感显示、隐藏面处理、阴影和着色，我们都需要把“边界表示”转成“面内像素表示”。这一步就是本章要解决的核心。

你可能会疑问：前面的绘制轮廓好像比较复杂，那这里填充不该很简单了吗？按照前面的 $F(x, y)$ 判别式，我们直接在包围盒里逐像素判断“这个点是否在多边形内还是外”，不就行了么？虽然这个方法概念清晰，但它的计算量非常大。因为对于一个 $m \times n$ 的包围盒，我们需要进行 $m \times n$ 次点内外判断，效率非常低下。因此，我们能否设计出一种方法，利用多边形边界的连贯性来减少重复计算呢？像扫描仪一样，一行一行地确定哪些区间属于图形内部？这就是扫描线算法的核心思想。

要学习这一点，首先我们先区分多边形的两种表示。第一种是**顶点表示 (vertex representation)**，也就是保存顶点序列。比方说有个四边形，那么我们就只需要存四个顶点的坐标和它们的连接关系就可以了。它的好处是结构清晰、内存占用小、做几何变换方便。但是仅仅知道这些信息，我们无法立刻知道哪些像素在多边形内部，哪些在外部。第二种是**点阵表示 (raster representation)**，也就是直接保存内部像素集合。它显然内存占用大，但我们可以直接知道哪些像素需要点亮，适合直接显示。

扫描转换本质上就是把前者转成后者，核心问题也就是：如何根据顶点表示，就高效地确定哪些像素在多边形内部，进而形成点阵表示呢？

接下来，我们就要引入**扫描转换 (scan conversion)** 这个关键词。它本质上就是按某种规则确定哪些像素在图形内部，然后把这些像素写入帧缓冲。现在硬件里，GPU 对三角形支持最成熟，所以复杂多边形常先三角剖分，再逐个三角形处理。但算法思想并不只限于三角形，它可以推广到一般多边形。

这里，我们先学习一种最直观的方法：**逐点判断法**。它的做法非常直接，在包围盒里逐像素检查“这个点是否在多边形内还是外”。判断点内外最常见是射线法：从待判点发一条射线，看和多边形边界交点个数 k 的奇偶性。奇数通常判内，偶数通常判外。还有累计角度法，思想是把点和各顶点连线形成的有向角求和，若总和是 $\pm 2\pi$ 则在内部，若接近 0 则在外部。这个方法概念清晰，很适合入门，但工程上很少直接用，因为每个像素都独立判断，计算量会非常大。

真正的高频考点是**扫描线算法**。它不再进行逐点的孤立判断，而是充分利用连贯性。简单说就是：同一条扫描线上，像素内外状态是成段变化的；相邻扫描线之间，交点位置变化也可以递推。这就让算法能从很多次的重算变成少量的更新。

扫描线算法的本质，就是通过“穿过边界的次数”来判断内与外。我们使用一个扫描线从上到下逐行横着扫描，从左到右，穿过 1 次边线就到达了内部，穿过 2 次就回到了外部，以此类推。通过这种方式，我们就可以在扫描线的每一行上，确定哪些区间是内部像素，哪些是外部像素，从而进行填充。

想要彻底掌握扫描线算法，我们要记住三个容易考的规则。

1. 交点取整规则：当交点落在两个像素之间时，我们要按约定取整，保证填充不会跑到边界外。也就是说，宁可往里缩一点，也不要往外扩一点。
2. 区间取舍中，通常采用左闭右开。比方说一个矩形的右边界是 $x = 10$ ，另一个矩形的左边界也是 $x = 10$ ，如果都取闭区间，那么 $x = 10$ 这一列像素就会被两个矩形重复填充，导致重叠区域的颜色异常。通过采用左闭右开，我们就可以确保边界只由右边的矩形负责。
3. 第三是顶点交点处理，也就是常说的“下闭上开”思想，避免一个顶点被重复计数导致交点奇偶性出错。在扫描线算法中，我们通过穿过边界的次数来判断内与外，但比如说三角形的顶点，我们应该算穿过了几次呢？为了避免误判是否进入内部、需要开始涂色，我们就约定：每条边只包含它的下端点，不包含它的上端点。通过这种约定，就可以解决扫描时所有遇到顶点的情况。

具体来说，在扫描线横向穿过一个顶点（两边交点）时，它可能是极值点（波峰波谷，此时两边都在同一侧，不应当开始涂色）或者非极值点（两边在不同侧，应当开始涂色）。那么实际中，我们可以如何判断呢？

不难发现，顶点 P_i 若满足

$$(y_{i-1} - y_i)(y_{i+1} - y_i) \geq 0$$

则它是极值点（其中 y_{i-1} 、 y_i 、 y_{i+1} 分别表示三角形的三个顶点的 y 坐标）。极值点常按两个交点处理，非极值点按一个交点处理。如此就能保证正确判断内外状态。

扫描线算法之所以高效，还因为它利用了边的连贯性。设当前扫描线和某条边交点横坐标是 $x_{d,r}$ ，下一条扫描线交点是 $x_{e,r}$ ，若边斜率是 m_r ，则常见递推关系是：

$$x_{e,r} = x_{d,r} + \frac{1}{m_r}$$

这就使得交点无需每次从头算，而可以基于上一次交点增量更新。

若对象本身就是矩形，那么扫描转换还可以进一步简化，因为矩形边与扫描线的交点规律非常规整，往往只需确定左右边界后逐行填充即可。因此我们常常会把矩形单独拿出来讲，它并不是一种独立的思想，而是多边形扫描转换里最简单、最适合先建立直觉的情形。

到实现层面，有两张表需要我们先了解：**边表 ET (Edge Table)** 和 **活动边表 AEL (Active Edge List)**。在实际问题中，边的数量可能很多，如果每次扫描线都要遍历所有边来找可能的交点，效率会很低。显然每次，我们只需要关心那些当前扫描线会穿过的边就好了。与当前扫描线相交的边，就称为**活性边**。因此，我们先将所有边按照它们的起点 $y_{\min}$ 进行排序，存入全集边表 ET 中。每当扫描线到达某个 y 坐标时，我们就把 ET 中所有起点在该 y 的边加入活动边表 AEL 中。而每次扫描线向上移动一格时，我们就删除那些在 AEL 中且当前高度已经超过了 $y_{\max}$ 的边。如此，AEL 中就会只包含当前扫描线会穿过的边，那些不相关的边我们就不需要考虑了。

因此，每推进一条扫描线，基本流程就是：把 ET 里该行新出现的边加入 AEL；按交点横坐标排序；两两配对（即按照奇偶性）填色；然后删掉到达 $y_{\max}$ 的边；再进行增量更新，更新交点 $x \leftarrow x + \Delta x$ 进入下一行。这个流程是本章最值得熟练描述的流程之一。

扫描线算法之后，我们再简单了解几种其他实现路线。例如，边缘填充算法就是不管三七二十一，遇到边就反转。也就是说，扫描线从左到右走，遇到边界就将右侧的所有像素都翻转黑白。这个方法虽然简单，但效率不高，因为它会对同一像素进行多次翻转，尤其是当多边形边界复杂时，I/O 操作会非常频繁。

栅栏填充算法是在边缘填充基础上的改进。它在多边形中间或者其他任意位置，引入一条与扫描线垂直的“栅栏线”，不再无脑向右取补，而是只在交点与栅栏之间取补，从而减少重复访问。

边界标志算法也很常考，它的本质也是基于交点奇偶性的原理。分两步：第一步，我们先沿着多边形的边界走一圈，把多边形边界像素打个标记；第二步就正常按扫描线从左到右走，维护布尔变量 `inside`。每遇到边界标记就翻转一次 `inside`。当 `inside` 为真时就填色。这个算法的优势是不用像之前的扫描线算法那样对交点进行排序，而是只需要顺序读写，所以在硬件实现里常常更有优势。

字符显示中我们常说的**轮廓字型法**，就和这里的边界思想很接近。它不再是像点阵字那样把每个字形直接存成像素块，而是用轮廓曲线或轮廓边界来描述字符，再在需要显示时进行填充。这样做的优点是缩放更自然、存储更经济，也更适合高质量排版与印刷。很多中英文字体系统，本质上都更接近这种“先描述轮廓，再做填充显示”的思路。

这几种方法在理解上不难，考题里一般更偏概念，不太会要求我们做复杂推导。

前面这些方法都偏“从边界出发”。而**区域填充算法** (region filling) 则是“从内部出发”。它先给一个种子点，再把颜色扩展到整个连通区域。就像在干涸的农田中开一个水龙头（种子点），水会向四周流淌，直到遇到田埂（边界）为止。这里，我们一定要先理解连通性，通常分 4 连通和 8 连通。4 连通只能往上、下、左、右进行扩展，而 8 连通还可以像四个对角方向扩展。连通定义不同，会直接影响算法的传播路径和边界要求。

经典种子填充 (flood fill) 一般用栈或递归实现。操作是：弹出一个种子点，着色，再把满足条件的邻点压栈。它直观、易写，还可以填有孔区域。但它的问题也很明显：递归层数深、进出栈频繁，区域大时效率和内存压力都会上来。而且很关键的是，格子 A 很可能既是格子 B 的上面，又是格子 C 的左面，这样就会被重复访问多次，效率更低。

因此在这里，我们给出扫描线种子填充的方法作为改进。它不再一个像素一个像素地扩散，而是一次先填完整个连续区段，再去上下两行找新区段种子。从某个点 (x, y) 开始，分别向左以及右扩展，直到遇到边界为止，然后就可以将这个区段的像素一次性填充完毕。接着，我们再检查这一行的上方和下方，看看是否有新的种子点需要继续扩展。如此，每个区段通常只需入栈一次，就明显减少了递归和重复访问。

在本章最后，我们再比较一下前面所述的“扫描转换”和“区域填充”，在简答题里很常见。需要抓住三点：

1. 目标相近但起点不同，扫描转换从边界顶点信息出发，区域填充从种子点出发。
2. 扫描转换会改变表示方式（顶点表示到点阵表示），而区域填充一开始就已经是点阵表示了，它不改变表示方式，只负责给区域内部的点阵着色。
3. 第三，边界要求不同，扫描转换强调交点配对正确，区域填充强调边界封闭，防止颜色外泄。

二者之间其实还可以相互转化。已知多边形顶点时，可以先用 DDA 或 Bresenham 画出边界，再从内部取一个种子点做区域填充；反过来，如果我们能稳定求出每条扫描线上的内部区间，那么本质上也完成了从边界表示到像素区域表示的扫描转换。把这层联系想通后，本章各种算法就不再是互相割裂的零碎技巧，而是同一个目标下的不同实现路线。

综上所述，本章高频出现、需要我们比较全面地了解的部分就是：扫描线算法流程、AEL/ET 含义与迭代、奇点处理原则、边界标志法思想、种子填充及其扫描线改进。

本节小练习

1. 简答题

为什么扫描线算法通常比逐点判断法快？请用一句到两句话说明核心原因。

2. 判断题

在扫描线填充中，扫描线与多边形边界交点个数可以是奇数，不影响配对填充。（____）

3. 简答题

请简要说明边界标志算法中 `inside` 变量翻转的意义。

参考解答

1. 因为扫描线算法利用了区域、扫描线和边的连贯性，避免对每个像素重复做独立内外判定，计算量更小。
2. 错。扫描线填充依赖交点两两配对，交点计数和奇点处理错误会直接导致填充区间错误。
3. 每遇到一次边界就翻转一次 `inside`，表示扫描线从“外部”进入“内部”或从“内部”离开“外部”，据此决定当前像素是否填色。

第六章 裁剪、反走样与消隐算法

本节概要

这一章是从单纯的理解几何，到在屏幕上显示出来的关键工程环节。你会学到三件事：第一，裁剪解决“只画窗口内的部分”，它直接决定图形系统速度；第二，反走样解决“离散采样导致的锯齿和细节丢失”；第三，消隐解决“深度信息丢失导致的二义性”。读完后，你应该能清楚说明多种裁剪算法的思路与差异，理解走样产生的原因与反走样方法的代价，并比较几种消隐算法在速度和真实感上的取舍。

知识点讲解

在前面的第四章，我们学习了如何绘制直线、圆等基本图形的轮廓；而在第五章，我们学习了如何把多边形内部填充成像素区域。现在，我们要进入一个新的阶段：**裁剪 (clipping)**。既然我们已经能够画出完整的图形了，为什么还需要裁剪呢？因为在实际应用中，我们通常只关心视窗 (viewport) 内的部分，而视窗外的部分可能根本不需要显示。比方说在玩一个 3D 游戏，实际的场景可能非常大，但玩家只能看到其中的一部分。为了提高效率，我们就需要裁剪掉那些视窗外的部分，只保留视窗内的部分进行后续处理。

裁剪工作看似简单，却是图形系统的高频环节，因为几乎所有几何都要先裁剪再光栅化（扫描转换）。裁剪做得快，整个系统就快。裁剪做得慢，后面再快也补不回来。

这里，我们先学习**点裁剪**。它最简单，窗口左下角为 (x_L, y_B) ，右上角为 (x_R, y_T) ，则点在窗口内必须同时满足两个不等式：

$$x_L \leq x \leq x_R$$

$$y_B \leq y \leq y_T$$

那么，到了更复杂的场景，应该怎么办呢？例如，我们要考察一根线段，那么我们显然不可能去判断线段上每个点是否在窗口内，这样就又走向扫描线的老路了。也不可能直接根据线段的两个端点来判断，因为它们可能一个在窗口内，一个在窗口外，或者两个都在窗口外但线段穿过窗口。因此，我们就需要更厉害的算法来处理，这就是**复杂裁剪 (complex**

clipping)。复杂裁剪的核心思想是：把线段或多边形与窗口边界进行**求交**，得到新的线段或多边形，这些新的线段或多边形就完全在窗口内了。

直线段裁剪是复杂裁剪的基础，因为曲线与多边形都可以拆成线段处理。线段裁剪只有三种总体情形：完全可见、显然不可见、部分可见。提高效率的关键是尽快判定前两类，尽量少做“求交”这种成本高的操作。常见方法有直接求交、Cohen-Sutherland、中点分割、参数化裁剪以及 Liang-Barsky。

直接求交算法的思想很朴素，把线段和窗口边都写成参数形式，求出交点参数，再判断交点是否落在边界上。它概念清楚，但计算次数多。比方说，框所在的区域是 $[0, 1]$ ，但可能远在 $[100, 200]$ 的线段都要参与求交和判断，效率非常低下。因此，它不适合做高性能实现，通常只用来作为理解与推导的起点。

如何提升效率呢？接下来，我们介绍一种经典的线段裁剪算法：**Cohen-Sutherland**。它的核心思想是通过区域编码来快速判断线段的可见性。我们知道，窗口有四条边，我们将这四条边界分别延长，就得到了一个九宫格。我们的目标就是判断线段的两个端点分别在哪个格子里。因此，我们用四位二进制标记端点相对窗口的位，上下左右各占一位。任意一位为 1，表示端点在对应边界的外侧。例如，如果某个端点的编码为 1010，就表示它在这个窗口的左上方这个宫格里。

编码规则让我们可以直接得到两个判断：如果两端点编码都为 0000，说明完全可见；如果将两个端点的编码按位与后结果不为 0，说明两端点在同一外侧区域，线段必然完全在外。这两种情况，就可以直接得出结论，要么全画，要么则完全不用管这根线段了。只有剩下的情况才需要求交（求交时也可以由编码指示与哪条边相交，省时）。

这一套逻辑能显著减少求交次数，所以是工程上常用的线段裁剪方法。

而**中点分割算法**更像用二分搜索来找交点。它先判断整条线段是否可能可见，若可能，就不断求中点。若从端点到中点的线段不是显然不可见，则交点一定在该子段上，用该子段替换原线段。否则就换到另一半继续分割。重复这个过程，直到子段长度小于阈值，中点就收敛到交点。它主要用加法和除法，适合硬件或移位优化。

参数化裁剪中，我们先把线段写成参数形式：

$$x = x_0 + t\Delta x$$

$$y = y_0 + t\Delta y$$

$$0 \leq t \leq 1$$

显然，这里的 $t = 0$ 对应线段起点， $t = 1$ 对应线段终点。这个 0 到 1 的参数区间就限定了线段的范围。如此，我们就可以把“求交”转为“求参数区间”。

Liang-Barsky 就是矩形窗口下的高效参数法。根据上述的参数表达式，我们不难发现，线段处在窗口内的条件就是要同时满足

$$x_L \leq x_0 + t\Delta x \leq x_R$$

和

$$y_B \leq y_0 + t\Delta y \leq y_T$$

这两个式子。经过变形，就可以得到有关 t 的四个不等式。我们只需要算出相关参数，随后进行 4 次除法，就可以得到这 4 个 t 的阈值。最后，只需要取最大下限与最小上限，就可以得到可见区间 $[t_l, t_u]$ 。当 $t_u > t_l$ 时线段可见，否则不

可见。这种方法在矩形窗口下比 Cohen-Sutherland 更少求交。

Cyrus-Beck 方法则面向凸多边形。这里所说的多边形，不是说要裁剪的对象是多边形，而是说我们窗口是一个凸多边形，不再是简单的矩形了。此时，我们依旧要判断这条线段是否在窗口内部。

设边界上一点为 A ，该点处的内法向量为 N 。在我们考察的线段上，假设某点为 $P(t)$ ，那么我们画图就不难发现，如果 $P(t)$ 在边界内侧，那么必须满足 A 到这个点的向量与 N 的点积为正。

也就是说，点 $P(t)$ 在凸多边形内的充要条件是：对于凸多边形所有边界上任意一点 A 和该点处内法向 N ，都有：

$$N \cdot (P(t) - A) > 0$$

如此，我们就有了判断线段上某点在多边形内外的方法。然而我们最终要做的工作仍然是算出线段参数方程中的可见区间，也就是 t 的范围。不难写出参数方程：

$$P(t) = P_1 + t(P_2 - P_1)$$

此时，我们即可将 $P(t)$ 代入上述的点积式，得到：

$$N \cdot (P_1 + t(P_2 - P_1) - A) > 0$$

我们将上式展开，即可得到 t 的一个不等式：

$$t > \frac{N \cdot (A - P_1)}{N \cdot (P_2 - P_1)}$$

但是需要注意，这里并非永远是大于的关系。一旦出现分母为负的情况，就会变成小于号。

也就是说，我们就可以利用上述的点积方法，对每条边求出 t 的一个阈值，而最后取所有边算出来的最大下限与最小上限即可确定可见区间。判断这些阈值是下限组，还是上限组的依据，就是 $N \cdot (P_2 - P_1)$ 的符号。比方说，如果内法向量和线段方向的点积为正，说明线段正在“从外部进入”，那么，得到的 t 显然就应该是一个下限。

因此，符号为正代表“向内进入”，属于下限组；符号为负代表“向外离开”，属于上限组。若线段与某条边平行，则只需判断线段在边界内侧还是外侧。

如果这个多边形就是矩形时，Cyrus-Beck 当然就会退化为 Liang-Barsky。

刚才我们看的是窗口为多边形，那么，如果我们要裁剪的对象不再是线段，而是多边形呢？比方说给出一个多边形，它的顶点有的在窗口内，有的在窗口外，我们要把它完全在窗口内的部分裁出来。这该怎么做呢？

此时，最重要的方法就是 **Sutherland-Hodgman** 逐边裁剪了。简单来说，就是拿这个多边形去依次“撞”矩形窗口的四条边（左、右、上、下），每搞一次，就能把当前多边形裁剪成一个新的多边形（如果有的部分被裁掉了的话），最后得到的就是完全在窗口内的部分了。这样操作的意义就是把“多边形对矩形窗口的裁剪”分解为“多边形对一条裁剪边的裁剪”：每处理一条边，都会生成一个新的顶点序列作为下一条边的输入。

那么每一“撞”时会如何裁剪呢？比方说，现在在“撞”窗口左边界，那么就需要对这个多边形的所有边顺次进行考察，看看是否需要裁剪。

此时，考察这条边的起点和终点是否在窗口内，只会出现四种情况：可见到可见（那么显然这条边全都在窗口内，输出终点即可），可见到不可见（说明终点跑到外面去了，应该输出交点），不可见到可见（那么说明起点根本就不在窗口内，真正的裁后起点应该是交点，故而输出交点和终点），不可见到不可见（全都不在，那完全无需输出任何点）。这样逐边裁剪后即可得到可见多边形。

但要注意，Sutherland-Hodgman 对凸多边形很稳定，对凹多边形可能出现“多余连线”。原因是输出只有一个顶点表，当裁剪结果被分成多个部分时（比方说“凹”这个字，从上面一刀砍去，会砍出来两个小方块），最后一个点会被自动连接到第一个点。常见解决方法是先把凹多边形分解成多个凸多边形分别裁剪，或使用 Weiler-Atherton 算法来维护多个输出边界。

接下来我们来看一个新的话题：**反走样**。我们知道，走样来自离散采样不足，高频信息被错误采样成低频，导致锯齿、摩尔纹与闪烁（比方说明明很平滑的曲线，因为低频采样而产生阶梯状或锯齿）。之前我们就学到，用奈奎斯特采样定理表述，就是采样频率 f_s 必须满足：

$$f_s \geq 2f_{max}$$

其中 f_{max} 为信号最高频率。

那么，如何反走样呢？常见方法有三类：提高分辨率、简单区域取样、加权区域取样。

提高分辨率最直观。若分辨率加倍，直线经过的像素数也加倍，阶梯更细，视觉上更平滑。但代价是帧缓存容量约增加 4 倍，扫描转换计算量也约增加 4 倍。还有一种软件方式是“先高分辨率计算，再低分辨率显示”，本质上仍是提高采样密度，只是把代价放到计算阶段。

简单区域取样的核心是“让像素有面积”。传统算法把像素当成点，线段当成零宽度直线，这与真实显示不一致。简单区域取样把线段看作有宽度的狭长矩形，计算其与像素方格的相交面积，面积越大亮度越高。为了简化计算，可以把像素细分为若干子像素（总数为 n ），统计落在线段内的子像素数 k ，近似面积为 k/n 。这相当于盒式滤波器（将图像中每个像素的值替换为其邻域像素的平均值，从而实现平滑、去噪等）。它的优点当然是简单，但缺点是忽略了相交区域的位置，容易造成相邻像素亮度跳变，比方说一个像素格子内，一开始无论白色是占据了左上角的十分之一，还是右下角的十分之一，最后对于这个格子都是一样的“漂白”。

而**加权区域取样**进一步引入权值，像素中心权重大，周围权值小。

在这里，我们给出两种常见的软件方法：一种把像素分成 4 个子像素，计算灰度后取平均；另一种把显示窗口分为 $(2m+1) \times (2n+1)$ 子像素。这样的主要目的是使得能够有一个中间像素。比方说最常见的情况 $m=n=1$ 时，就是对中心及周围 9 个子像素按权值表加权。这样能更平滑地过渡亮度，但计算量也随子像素数量增加。

最后，我们来看本章的最后一个主题：**消隐**。我们在之前的章节学习中就知道，透视投影会丢失深度信息，导致“线框二义性”。比方说在中学，我们都学过如何画一个立方体，显然有的边要连实线，有的则要连虚线，才能表达出哪个在前，哪个在后。但计算机并不知道这件事情。如果不根据深度信息来判断，那么万一计算机先画了前面的面，再画后面的面，结果就会把后面的面的信息覆盖到这些像素上去了，这显然是错误的。

消隐就是决定什么东西可见。消隐对象可以是边，也可以是面。在这里，我们主要学习**面消隐**，按处理单位可以分为**图像空间算法**与**物体空间算法**。前者以像素为单位，例如 Z-Buffer 与扫描线 Z-Buffer；后者以物体为单位，例如光线投射。

两类算法的复杂度差异明显，图像空间通常是 $O(mnkh)$ 级别（这里的 m 表示屏幕宽度， n 表示屏幕高度， k 表示多边形数量， h 表示扫描线数量），而物体空间通常是 $O((kh)^2)$ 级别。实际使用时会根据场景规模和实时要求做取舍。

画家算法处在消隐的入门位置。它的思想很像画画时“先画远处背景，再画近处物体”：把各个面按与视点的远近排序，远的先画，近的后画，后画的把前面的覆盖掉。这个思路直观，但遇到面之间循环遮挡、相互穿插时就会出问题，因此更适合作为理解消隐思想的起点，而非最稳妥的通用方案。

Z-Buffer 的思路是为每个像素维护一个深度值，若当前多边形在该像素处深度更小，就更新像素颜色与深度。它与绘制顺序无关，简单稳定，易于硬件实现。标准流程是：帧缓存置背景色，深度缓存置最小值。如此，当我们遍历每个多边形时，就可以计算它覆盖的像素深度，并与缓存比较，若更近则需要更新。

若多边形平面方程为 $ax + by + cz + d = 0$ ，则深度即可写为：

$$z = -\frac{ax + by + d}{c}$$

当然，它的缺点也比较明显：需要额外深度缓冲，并且每个被多边形覆盖的像素都要计算深度。

还有一个只用一个深度变量的改进版思路。它把处理单位从“多边形”变成“像素”。做法是对每个像素遍历所有多边形，只保留深度最大的那个面。这个方法确实减少了深度缓存空间，但计算量更大，因为每个像素都要遍历所有多边形。还是没有解决本质问题。

扫描线 Z-Buffer 把 Z 缓冲缩减为一条扫描线大小，并借助多边形分类表与活化多边形表减少无效求交。原先的 Z-Buffer 算法中，我们要时时刻刻记住每个像素的深度信息，这是因为任何一个多边形都可能随时覆盖到这个像素上去。但是，在之前章节中我们就学到过，扫描线算法的精髓就在于引入了分类表。比方说先按照 y 坐标排序，每当扫描线到达某个 y 坐标时，只把该行新出现的多边形加入活化多边形表中。这样，我们就只需要关注当前扫描线会穿过的那些多边形。同时，我们也不需要为每个像素维护一个深度值了，之前要维护是因为要遍历所有多边形，然而现在我们只需要维护当前扫描线穿过的那些多边形的深度信息就好了。具体实现时，我们仍然可以记录深度增量，如此就可以利用连贯性增量计算深度，减少每像素的重复运算。

扫描线 Z-Buffer 常会配合三类表结构一起出现：**多边形分类表 PT**、**活化多边形表 APL** 和 **活化边对表 AEPL**。它们的作用分别是把多边形按扫描线起始位置分类、记录当前扫描线真正相关的多边形，以及记录这些多边形在当前扫描线上的边对信息。即使不要求你把数据结构细节全部背下来，也最好知道这些缩写分别在做什么。

区间扫描线算法再进一步，不再对每个像素计算深度，而是按区间决定可见面。在一条扫描线上，将多边形的所有交点排序后形成多个区间，那么大概率这个区间内的深度顺序是相同的。因此，我们就可以取区间内任意一点计算深度，深度最大的多边形就直接填满整个区间。它的优点是减少深度计算、速度更快了，但也要求多边形不相互贯穿，否则同一区间内深度顺序可能发生交换。

除此之外，还有**区域子分割算法**和**四叉树消隐**这类基于区域连贯性的思路。它们的共同点是：先不急着重对每个像素都做精细判断，而是先看一个较大区域是否“足够简单”。如果整个区域被同一面完全覆盖，或者区域内可见关系已经明确，就可以直接显示；只有在情况复杂、无法直接判断时，才继续把区域细分。四叉树方法就是不断把二维区域分成四个更小子块，直到每一块都简单到可以直接处理为止。它的优点是能利用大块区域的一致性来减少无谓计算，但场景过于复杂时，细分层次也可能很多。

光线投射算法从每个像素发射投影线，与场景多边形求交，取最近交点决定颜色。可以理解成从眼睛出发，沿着视线方向发射一条光线，看看它首先碰到哪个多边形，就把那个多边形（深度最小）的颜色赋予该像素。它属于物体空间算法，真

实感更高，但计算量也最大。实际系统总是在速度与真实度之间取折中，实时模拟倾向快速算法，离线渲染倾向高真实度算法。

本节小练习

1. 简答题

简述 Cohen-Sutherland 裁剪中“区域编码”和“按位与”的作用。

2. 判断题

提高分辨率是消除走样的最理想方法且代价很小。（_____）

3. 简答题

请用一句到两句话说明 Z-Buffer 的基本思想，并指出它最大的代价是什么。

参考解答

1. 区域编码用于快速判断端点相对窗口位置，按位与不为 0 表示两端点同在某一个外侧区域，线段必不可见。
 2. 错。提高分辨率能减轻锯齿，但显存与计算量显著增加。理想的方法应当是根据场景和性能需求进行合适的反走样。
 3. 每个像素保存最接近视点的深度值，遇到更近的面就覆盖颜色。代价是需要深度缓冲并进行大量像素级深度比较。
-

第七章 真实感渲染技术

本节概要

这一章要解决的问题是：为什么有些图形只像“被画出来的多边形”，而有些图形会让人觉得像一个真的物体被灯光照亮、被环境包围、并且确实存在于空间中。要回答这个问题，我们需要沿着三条主线来理解。第一条是**光照模型**，也就是物体表面在光照下为什么会出现明暗、颜色、高光与材质差异。第二条是**网格着色技术**，也就是当真实曲面被许多三角形近似之后，怎样把本来离散的多边形显示得尽量光滑自然。第三条是**全局光照与阴影**，也就是除了直接照明之外，反射、折射、透明和遮挡为什么也会深刻改变图像结果。

学完这一章后，读者应当能够清楚地区分局部光照与全局光照，理解环境光、漫反射、镜面反射在视觉上分别对应什么效果，掌握 Phong 光照模型的核心结构，理解 Gouraud 着色与 Phong 着色的差异，知道 Whitted 模型与光线跟踪的基本递归思路，并能够说明阴影图与阴影体这两类经典阴影算法的基本原理、优点和局限。

知识点讲解

我们先从一个最根本的问题开始。前面几章中，我们已经学会了如何表示几何对象、如何把它们投影到屏幕上、如何裁剪、反走样与消隐。到了这一步，一个三维场景其实已经可以“显示出来”了。但仅仅能够显示，并不意味着看起来真实。假设屏幕上有一个球，如果我们只画它的轮廓，或者把它整个涂成同一种颜色，那么它虽然有形状，却很难让人感到它真的具有体积、材质和空间位置。真实感渲染要做的，就是把几何形体进一步转化成一种有光照、有层次、有遮挡关系、有材料感觉的图像。

要理解这件事，先要明白观察者最终看到的颜色并不是物体“自己带着”的，而是多种因素共同作用的结果。物体的几何形状会影响法向方向，从而影响它如何接收光；光源的位置、距离、颜色、强度和类型会影响入射光；周围环境中的遮

挡、反射和折射会带来间接照明；观察方向又会决定某些高光是否能被看到；而材料本身还决定了物体更像粉笔、塑料、金属，还是玻璃。这也是为什么真实感渲染从来不只是“给模型涂颜色”，而是要综合考虑物体、光源、环境和视点。

接下来，我们先区分一个贯穿整章的重要概念：**局部光照模型**和**全局光照模型**。局部光照模型只考虑“光源直接照到当前表面”这件事，也就是说，它关注的是光源到物体表面的直接贡献，而忽略其他物体的反射、折射和二次照明。这样的模型通常计算快、结构清楚，是实时渲染中的基础。全局光照模型则进一步考虑光在场景中的传播与多次作用，例如一束光先照到桌面，再从桌面反射到墙面；或者光线经过玻璃折射后再进入视线。这一类模型更接近真实世界，但代价也更高。

在正式写光照公式之前，我们先把光源本身说明白。按几何形状区分，常见有点光源、线光源和面光源。点光源最容易处理，它可看成空间中的一个点，向四面八方发光，所以很多基础模型都从点光源出发。按光谱组成区分，又有白色光、彩色光和单色光。白色光可以理解为多种可见波长的组合，彩色光则更偏向某些波段。由于显示设备通常采用 RGB 三基色表示颜色，所以实际渲染时我们往往会分别对红、绿、蓝三个通道建立光照计算，再组合成最终颜色。

在建立光照模型之前，还需要一点基础物理知识，这些我们大多在中学就已经学过。首先是**反射定律**：入射角等于反射角，入射光线、反射光线和表面法向在同一个平面内。其次是**折射定律**，它描述光从一种介质进入另一种介质时方向如何变化，常写为

$$\eta_i \sin \theta_i = \eta_t \sin \theta_t$$

这里， η_i 和 η_t 分别是入射介质与透射介质的折射率， θ_i 和 θ_t 分别是入射角和折射角。这个公式在透明与折射问题里非常关键，因为它说明折射方向并不是随意选的，而是由材料的光学性质决定的。

除了方向规律，还要知道光是怎样被“计量”的。立体角衡量的是某个物体相对于视点张开的空间角度，它与二维中的平面角类似，只不过现在考虑的是空间中的张角。我们知道，在二维平面中，从一点看一小段圆弧，对应的平面角微元可被写为：

$$d\theta = \frac{dl}{r}$$

那么，类比到空间中，若一个面积元 ds 距视点距离为 r ，则对应的立体角微元可以写成

$$d\omega = \frac{ds}{r^2}$$

球面对观察点张开的最大立体角是 4π 。这个量在图形学里之所以重要，是因为许多与辐射相关的物理量都要按立体角来定义。与之相关的还有光通量和发光强度。光通量表示单位时间内通过某个面元的光能量，而发光强度则可以理解为单位立体角内的光通量。虽然在入门阶段不一定每次都直接计算这些量，但知道它们的物理含义，有助于理解“为什么渲染里的光照不是纯经验拼凑，而是有物理背景的”。

再进一步，光照过程中还满足**能量守恒**。一束入射光照到表面后，不会无缘无故变强。它的能量会分配到几个部分：一部分形成漫反射，一部分形成镜面反射，一部分可能透射进入物体内部，还有一部分被材料吸收并转化成熟。若把这些分量分别记为入射、漫反射、镜面反射、透射和吸收，对应的总关系可以理解为：入射能量等于这些分配项之和。这个结论虽然看起来朴素，但它提醒我们：光照模型必须有一定的物理合理性，而不能让表面凭空“制造出”额外亮度。

理解完这些背景之后，就可以进入本章第一条真正的主线：**简单光照明模型**。最初的局部光照模型通常把光照效果拆成三部分：环境光、漫反射和镜面反射。这种分法极其重要，因为它对应了我们视觉上最常见的三类现象。环境光负责给整

个物体一个基本亮度底色，解释为什么即使某处没有被强光正面打到，也不至于完全漆黑；漫反射负责表现“亮面和暗面”的大尺度变化，主要由表面朝向决定；镜面反射则负责表现高光和材质，解释为什么金属、塑料和玻璃会在某些角度显得特别亮。

先看**环境光**。如果一个场景中有很多物体，光会在它们之间反复散射。真正去精确计算这种多次反射是很贵的，所以早期模型常用一个简化想法：假设空间中存在一种近似均匀分布的背景光，不区分方向，也不显式追踪来源，这就是环境光。若环境光强度记为 I_a ，材料对环境光的反射系数记为 K_a ($K_a = 0$ 表示完全吸收，表面最暗； $K_a = 1$ 则表示完全反射，表面最亮)，则表面呈现的环境光亮度写为

$$I_e = K_a I_a$$

这里的 I_e 是表面最终贡献出的环境光分量。这个公式很简单，却非常有用。它说明不同物体即使处在同样的环境里，也可能因为 K_a 不同而显得有深有浅。不过，它的局限也同样明显：如果只靠环境光，那么同一物体表面各处亮度是恒定的，缺少明暗变化，自然就没有体积感。

接下来考虑最关键的那部分，也就是**漫反射**。粗糙表面会把入射光向各个方向比较均匀地散射出去，因此只要视线不是极端特殊，观察者看到的亮度主要取决于光线打在表面上的角度，而不太取决于观察方向。Lambert 模型正是沿着这一思路建立的。它的核心结论是：漫反射亮度与入射方向和法向之间夹角的余弦成正比。若光源强度为 I_p ，材料漫反射系数为 K_d ，入射方向与法向夹角为 θ ，则漫反射分量可写为

$$I_d = I_p K_d \cos \theta$$

若使用单位法向量 N 和单位光照方向 L 来表示，则因为 $\cos \theta = N \cdot L$ ，上式也常写成

$$I_d = I_p K_d (N \cdot L)$$

当然，这里通常默认当 $N \cdot L < 0$ 时取 0，因为光从表面背后照来时，不应给正面带来漫反射亮度。这个公式之所以重要，是因为它几乎决定了三维物体最基本的“亮面朝光，暗面背光”的视觉效果。

不过，仅有环境光和漫反射仍然不够。粉笔、石膏之类的粗糙材料还可以接受，但一旦遇到抛光金属、塑料或玻璃，我们就会发现它们表面常出现特别亮的斑块，这就是**高光**，它来自**镜面反射**。理想镜面反射中，只有恰好位于反射方向上的观察者才能看到反射光；稍微偏离，亮度就会迅速下降。真实材料当然没有这么理想，所以图形学中通常使用一个“非理想镜面反射”模型去近似这种现象。

在这套描述中，通常记 P 为表面点， N 为该点单位法向量， L 为从 P 指向光源的单位向量， V 为从 P 指向视点的单位向量， R 为理想反射方向单位向量。若入射方向与法向确定，则反射方向可写为

$$R = 2(N \cdot L)N - L$$

这条公式不难理解，因为本质上入射光线和反射光线是完全对称的，故而将法向量延长到两倍，再减去入射方向，就得到了反射方向。这条公式也非常值得记住，因为它直接体现了“把入射方向关于法向镜面对称”这件事。随后，我们就可以用视线方向与反射方向的接近程度来描述高光强度。Phong 镜面反射模型常写成

$$I_s = I_p K_s (R \cdot V)^n$$

其中, K_s 是镜面反射系数, n 是镜面反射指数。这个 n 很重要, 它控制高光的“尖锐程度”。如果 n 很大, 由于余弦值一定小于 1, 就会导致 I_s 很小。因此, 只有当 V 非常接近 R 时, 亮度才会明显, 因此高光会又小又尖, 看起来像非常平滑的抛光表面; 如果 n 较小, 高光分布会更宽, 看起来像表面较粗糙、散射较强的材料。

到这里, 三部分已经齐了。于是我们就得到了整章最核心的局部光照模型, 也就是 **Phong 光照模型**。它并非从零发明一种全新规律, 而是把环境光、漫反射和镜面反射三部分直接相加。若总亮度记为 I , 则可以写成

$$I = I_e + I_d + I_s$$

再把前面的表达代入, 就得到更完整的形式:

$$I = K_a I_a + I_p K_d (N \cdot L) + I_p K_s (R \cdot V)^n$$

这个模型在图形学历史上地位极高, 因为它第一次以很简单的代价, 把“底色、明暗、材质高光”三件事情统一到了一个公式里。对初学者来说, 理解它的关键就在于明确三项各自负责什么视觉现象。只要这个分工看清楚了, 后面很多算法其实都只是在思考: 怎样把这套局部模型更高效地应用到整片表面上。

不过, Phong 模型仍然只是一个经验模型, 它并不完美。首先, 它没有真正计算物体之间的相互反射, 所以环境光只能粗糙代替间接照明。其次, 它常会让物体看起来有一点“塑料感”, 因为高光颜色与材质关系刻画得不够细。再者, 在大入射角时它会出现一定失真。因此, 我们应当把它看成一个非常经典、非常实用, 但并非物理上完全严格的基础模型。

除了光照分量本身, 局部光照里还有一个容易被忽略却很实际的问题, 那就是**光的衰减**。点光源不是无论远近都一样亮的。随着距离增加, 光强应当减弱。理想点光源常按 $1/d^2$ 衰减, 但直接这么做有一个工程问题: 离得很近时变化过猛, 离得很远时变化又太慢, 所以实际系统里常会使用经过调整的经验衰减函数。这部分真正需要把握的是核心思想: 距离不仅会影响“有无被照到”, 还会影响“被照得有多强”。

与此相关的另一个技术叫**深度暗示** (Depth Cueing)。它最早用于线框图显示, 目的是让离观察者远的部分更暗, 近的部分更亮, 从而增强空间纵深感。虽然这不是真正的物理衰减, 但在视觉上很有帮助。常见做法是选定前后两个参考面, 给出对应的亮度比例因子, 再根据物体点的深度在这两个参考面之间做插值。这样处理后, 即使场景没有复杂雾效或大气散射, 远近关系也会更直观。

这两个面通常被称为近参考面远参考面。我们确定“绝对亮”的起点 (近参考面 d_{min}): 在这个面之前, 或者比它更近的物体, 衰减因子为 1, 即保持 100% 原色, 不衰减; 再确定“绝对暗”的终点 (远参考面 d_{max}): 在这个面之后, 或者比它更远的物体, 衰减因子达到设定的最小值, 比如 0 或 0.2)。在两个面之间, 我们就做线性过渡, 即只有处在之间的物体, 才会根据它的深度 d , 在 100% 亮度和最小亮度之间做线性插值。深度暗示最早是给线框图 (Wireframe) 用的。这样的好处很明显: 比方说上面提到的线框图, 它没有光照和材质。如果我们直接用 $1/d^2$, 那当物体稍远一些时, 线条亮度就会断崖式下跌, 导致远处的结构直接“消失”; 而极近时, 亮度又会趋近于无穷大。

再往前走一步, 图像通常不是黑白的, 而是彩色的。于是我们就需要把光照模型扩展到 RGB 三个通道。这里的基本想法并不复杂: 对红、绿、蓝三个通道分别做环境光、漫反射和镜面反射计算, 再把结果组合起来。材料颜色可以通过不同通道上的反射系数来控制。例如红色物体并不是“只会发出红光”, 而是它对红色分量反射更强, 对绿色和蓝色分量反射更

弱。多个光源的情形也类似，总亮度就是各个光源贡献的叠加。这意味着：一个场景中，光源数量增加不仅会让画面更亮，更重要的是会使亮度分布、高光位置和综合色调都发生变化。

到这里，我们已经能够在一个点上算出光照了。但现实问题是：屏幕上显示的不是“一个点”，而是一整个多边形网格。于是本章的第二条主线就出现了：**网格着色技术**。我们知道，在三维图形中，复杂曲面通常不是直接以真正的连续曲面显示，而是由大量小三角形或小多边形去逼近。这样一来，即使每个小面本身都不大，只要处理方式不对，人眼仍然会看到明显的棱角和拼接痕迹。着色技术要解决的，正是怎样把离散网格看起来变得更加平滑自然。

到这里，读者可能先会有一个疑问：在计算某个点的光照时，需要该位置法向量的参与。在前面的模型讨论中，“法向量”好像都是直接现成给出的，那么在真实场景的多边形渲染中，我们怎么知道一个点的法向量呢？这并不难做到。以三角形为例，在图形学中，场景一定会先给出各顶点的坐标。只要有坐标，我们就能很轻松地得到两条边向量 $\vec{AB}$ 和 $\vec{AC}$ 。在中学，我们就学过：对这两条边做向量外积（叉乘）后得到的，就是完全垂直于这个三角形平面的垂线。把它单位化后，就是之前模型中所需要的单位法向量了。

接下来，我们就来看看网格着色有哪些常见的技术。

最直接的方法叫**平坦着色** (Flat Shading)，也叫均匀着色。它的思路极其简单：对一个多边形只计算一次颜色，然后用这个颜色填满整个面。这样做的优点当然是快，因为每个面只算一次光照；但缺点也非常明显。相邻多边形的法向通常不同，所以它们算出来的颜色会突变，结果就是明暗在边界处突然跳变，整个物体看起来像一块一块的刻面体。这种方法只有在多边形本来就代表真实平面，或者光源、视点都足够远、颜色变化很小的时候，效果才勉强可以接受。

为了让过渡更平滑，图形学引入了**插值着色**。这里最经典的两种方法分别是 Gouraud 着色和 Phong 着色。二者的共同点都是：不再简单地把一个面当成“只有一个法向、一个颜色”的整体，而是试图把顶点信息延伸到多边形内部，争取让多边形内的颜色变化更平滑。这两种方法的核心差别就在于“插值的到底是颜色，还是法向量”。

先看 **Gouraud 着色**。它的基本流程分四步：

1. 先计算每个多边形的单位法向量。
2. 求每个顶点的近似单位法向量。我们知道，理论上在各多边形的交界处，所谓的法向量是不存在的。但是，我们可以把与该顶点相邻的多个面法向做平均。这样得到的平均法向，往往就很接近原始光滑曲面在该顶点处的切平面法向。
3. 在每个顶点处使用光照模型算出顶点颜色。
4. 再把顶点颜色沿边和扫描线插值到多边形内部，即把顶点的颜色按距离按比例地分摊给内部的像素。于是整个面上的颜色就不再是突变的，而是连续变化的。

这一方法的优点非常直接：它比平坦着色平滑得多，而且计算量仍然比较低，因为真正昂贵的光照方程只在顶点上算，而不是在每个像素上算。因此，它对漫反射主导的物体效果很好，也很适合早期硬件和资源有限的系统。但它有两个典型问题。第一，高光容易丢失或位置异常。原因是高光本来可能出现在三角形内部，但顶点上并不亮，那么经过颜色插值后，这个高光就可能被“抹掉”。第二，它还会出现**马赫带** (Mach band) 效应，也就是人眼会在亮度梯度变化处感到不自然的明暗条纹。

于是就出现了改进方案：**Phong 着色**。它与 Gouraud 的最大区别是，不再在顶点上先算好颜色再对其插值，而是改为了直接“对法向量做插值”。我们先算出顶点法向量，接着就可以把多边形内部每个像素对应的法向量按照插值近似出来，然后在每个像素处再使用光照方程计算颜色。也就是说，Gouraud 是“插值颜色”，Phong 是“插值法向量，再算颜色”。这样一来，镜面反射和高光的位置就能在像素级别更准确地表现出来，因此视觉上通常更真实。

不过，Phong 着色的代价也很明确：计算量显著增加。因为原本只在顶点上算一次光照，现在变成对每个像素都要进行法向插值和光照计算。换句话说，它是用时间换效果。对于高光明显、材质变化敏感的场景，这个代价通常是值得的；而

对很多以漫反射为主的场景，Gouraud 依然是一个很有吸引力的折中方案。两种方法并没有简单的优劣之分，而是在效果和性能之间做不同权衡。

讲完着色之后，本章还需要讨论一个很容易和光照混在一起的问题，那就是**透明**。在现实世界里，很多物体并不是完全不透明的，例如玻璃、水、塑料片等。透过它们，我们不仅能看到它们本身的高光和表面颜色，还能看到后面的景物。这种现象如果完全不考虑，很多真实感场景就会明显失真。

最简单的透明模拟，实际上并不去认真处理折射，只是假设光线穿过透明体时方向不变。这样虽然不够真实，但在很多场合非常实用，因为它避免了景物因折射而变形，同时又能呈现“看得过去的透明效果”。最常见的思路是把前景透明物体和后景不透明物体的颜色按透射系数做混合。若前景颜色记为 I_1 ，背景颜色记为 I_2 ，透射系数记为 K_t ，则一种典型的插值形式可以写成

$$I = (1 - K_t)I_1 + K_tI_2$$

当 $K_t = 0$ 时，说明前景完全不透明；当 $K_t = 1$ 时，说明前景完全透明。这个式子虽然简单，但已经足以描述很多基础透明现象。

如果进一步考虑材料对不同颜色的透射差异，就会得到所谓的过滤透明。此时透明物体可以看作一个颜色滤波器，它不会平均地放过所有光，而是更容易透过某些颜色，抑制另外一些颜色。这也解释了为什么彩色玻璃透过后的景物会带有特定色调。再进一步考虑折射时，问题就变得复杂得多，因为透射光线方向会随着法向和折射率变化而偏折。特别是当光从高折射率介质射向低折射率介质，且入射角足够大时，还会出现**全反射**。这时折射光线消失，全部能量转到反射方向上去。

到这里，我们就可以自然过渡到本章第三条主线，也就是**全局光照模型与光线跟踪**。前面的局部模型虽然已经能产生不错的结果，但它始终没有真正处理“光在场景中的传播”。例如镜子里为什么能看到别的物体？玻璃后面的物体为什么会发生折射偏移？墙面的颜色为什么会反射到附近物体上？这些都不是单靠环境光加漫反射加镜面反射就能完全解释的。于是，图形学需要一种能显式考虑反射与折射路径的模型。

最经典的入门模型就是 **Whitted 光照模型**。它假设视点方向上看到的亮度由三部分组成：第一部分是光源直接照射后在该点产生的局部反射亮度；第二部分是来自镜面反射方向上的其它物体贡献；第三部分是来自透射方向上的其它物体贡献。若把这三项分别记为 I_{ll} 、 I_{sl} 和 I_{tl} ，并用 K_s 和 K_t 控制反射、透射贡献，则有

$$I_l = I_{ll} + K_sI_{sl} + K_tI_{tl}$$

这里的关键在于：后两项不是在当前点局部就能算出来的，而是必须沿反射方向和透射方向继续去场景里寻找别的物体，看看那些方向上究竟“看到”了什么。于是，这个模型天然就是递归的。

这也就引出了**光线跟踪** (Ray Tracing) 的基本思想。自然界中，光线是从光源出发，经场景反射或折射，最后进入人眼的。但如果真的按这个过程去模拟，会非常浪费，因为绝大多数光线最终并不会进入眼睛。于是图形学采用了一个非常聪明的逆向思路：从视点出发，向每个像素发射一条视线，看看它首先与哪个物体相交，这个最近交点就是该像素对应的可见点。接下来，在这个点上先用局部光照模型算出直接照明，再沿镜面反射方向和透射方向继续发射新光线，递归地收集来自其它物体的贡献。这就是 Whitted 风格光线跟踪的核心。

如果把它整理成流程，其实非常清楚。对于屏幕上的每个像素，先构造从视点穿过该像素中心的射线。然后求这条射线与场景中所有物体的最近交点。若没有交点，则这个像素通常显示背景色；若有交点，则先求局部亮度。接着，如果允许继续递归，就构造反射光线；如果物体透明，还构造透射光线。然后分别递归求得它们返回的亮度，再按 Whitted 模型合

成。可以看出，光线跟踪并不是神秘复杂的黑箱，而是“可见性交点 + 局部光照 + 递归反射透射”三件事的组合。

当然，递归不能无限进行下去，所以一定要有**递归终止条件**。最常见的终止条件有三个。第一，光线没有与任何物体相交，此时直接返回背景。第二，递归层数已经达到上限，因为不可能无限反射下去。第三，当前这条光线对最终像素亮度的贡献已经小到可以忽略不计，例如经过多次乘上 K_s 和 K_t 之后，亮度衰减到某个阈值以下，就停止追踪。理解这三条条件非常重要，因为它们直接体现了光线跟踪在精度与效率之间的平衡方式。

从效果上看，光线跟踪最大的优势是它天然适合处理镜面反射、折射和阴影，因此能够比较方便地生成具有较强真实感的图像。它的主要劣势则是计算量大。因为每一条射线都要与场景求交，而每个交点处又可能继续分裂出反射光线和透射光线，射线数目会快速膨胀。所以在没有专门优化和硬件支持的年代，它更适合离线渲染而非实时绘制。

说到这里，就必须单独把**阴影**拿出来讲。因为即使不看材质、不看高光，仅凭阴影本身，人的视觉系统就已经能获取大量空间信息。一个球如果没有阴影，我们往往很难准确判断它是贴在地面上，还是悬在空中；而一旦给出正确阴影，它的位置关系立刻就清楚了。这说明阴影不仅仅是“附加特效”，而是空间感和遮挡关系的重要来源。

从定义上说，如果场景中的一点完全看不到光源的任何部分，那么它位于**本影** (umbra) 中；如果它只能看到光源的一部分，那么它位于**半影** (penumbra) 中。如何形象理解呢？关键在于：光源是有体积的，还是一个无限小的点。

本影可以理解成绝对的黑暗。比方说在阳光下，我们把手紧贴在桌面上，那么手正下方那一块最黑、边界最清晰的阴影，就是本影。如果我们使用一个无限小的点光源，那么它产生的阴影只有本影，没有半影。而半影则可以理解成“半明半暗的过渡带”，因为遮挡物只挡住了光源的一部分。比方说，我们把手从桌面抬高一点，就会发现影子的边缘变得模糊了。那个模糊、发虚的毛边，就是半影。这正是因为光源发出的部分光被手挡住了，但是仍有部分光漏过来，只不过不够亮。在现实中的光源，如日光灯、太阳，都是有体积的光源。只有计算了半影，渲染出来的影子看起来才高级、真实。本影和半影合在一起，就构成了我们通常说的阴影。

此外，挡住光源的物体叫遮挡物 (occluder)，接收阴影的表面叫接收者 (receiver)。把这些名称分清楚很重要，因为后面很多阴影算法讨论的，其实就是“谁挡住了谁，谁又在接收阴影”，也就是下面所说的阴影类型。

在阴影类型上，我们常将阴影分为附着阴影、投射阴影和自阴影：

1. 附着阴影可以理解为物体自身由于朝向背光而形成的暗部，此时的遮挡物和接收物都是物体自己。比如白天地球的夜半球；或者一个苹果，朝向窗户的那面是亮的，背向窗户的那面是黑的。在计算时，本质上就是局部光照里的法向与光照方向关系。只需用到前面提到过的 $N \cdot L$ ，当点积小于 0，说明光照不到背面，直接涂黑即可。
2. 投射阴影则是某个物体挡住光后，把阴影投到别的表面上，此时的遮挡物和接收物是两个不同的物体。比方说你在路灯下走，你的身体挡住了光，但影子呈现在了马路地面上。此时就必须显式做可见性判断了，需要用到著名的阴影图或阴影体算法，判断光线从路灯出发，是不是被身体截断了。对于这两种算法，在下面会详细介绍。
3. 自阴影则是一种特殊的投射阴影，也就是一个物体挡住自己的部分区域。例如一个复杂模型的某个凸起结构，可能把后面的凹陷区域遮住。其本质也是投射阴影，计算方法也十分类似。

把这三者区分清楚，有助于我们理解为什么有些阴影只靠光照模型就能表现，而有些必须显式做可见性判断。

继续往下，需要区分**硬阴影**和**软阴影**，它们和之前的本影、半影的概念密切相关。如果把阴影看成一个二值状态，即某点要么在阴影里，要么不在，那么得到的就是硬阴影。它通常对应点光源，边界很锐利，计算也比较方便。但现实中的大多数光源都有一定面积，例如面光源、灯管，甚至太阳在视觉上也并非严格的点。因此，实际阴影边缘往往不是瞬间从亮跳到暗，而是存在一段半影过渡区域，这就是软阴影。软阴影更真实，但因为它要求处理光源不同部分的可见性关系，所以计算要困难得多。

在某些特殊情况下，阴影问题可以被大大简化。例如当阴影投射到一个平面上时，就得到**平面阴影**。为什么要专门考虑这

一点呢？前面我们提到，投射阴影计算起来非常麻烦、昂贵，因为计算机需要判断复杂的“可见性”。但是，如果接收影子的地方是一个“完美的平地”，我们就可以利用纯粹的初中几何（相似三角形）把这个问题快速计算出来。

它的核心思想是把遮挡物顶点沿着光源方向投影到接收平面上，从而得到阴影轮廓。若接收平面恰好是 $y = 0$ ，那么可以通过相似关系把空间点映射到该平面上的投影点；更一般地，如果平面是斜坡呢？在中学我们就学过，任何一个平面都可以用一个法向量 n 和一个常数 d 来表达，因此若平面方程写成

$$n \cdot P + d = 0$$

此时，类似地，只要有光源位置和斜坡的方程，通过向量投影公式，我们依然能找到空间中任意一点 v 沿光线射到斜坡上的落点 p 。而线性代数最擅长处理这种线性的映射，因此，我们可以把这一套复杂的投影计算，打包压缩到一个投影矩阵 M ，使空间点 v 的阴影投影满足

$$Mv = p$$

平面阴影之所以重要，不在于它覆盖范围最广，而在于它说明：阴影本质上也是一种几何投影问题。只不过一般场景中的接收面并不总是平面，因此还需要更通用的算法。

最后，我们来看目前最经典的两类通用阴影算法：**阴影体** (Shadow Volume) 和**阴影图** (Shadow Map)。它们的出发点完全不同，一个更偏几何体积思路，一个更偏图像深度思路。

先说阴影体。首先，我们先要理解什么是“阴影体”。如果从点光源出发，把遮挡物的轮廓边沿着背离光源的方向拉伸到远处，那么就会形成一个三维体积，这个体积内部的点都处在阴影中。这就是所谓的“阴影体”。比方说，关掉房间所有的灯，点燃一根蜡烛，再把一张名片挡在蜡烛前面。这时候，名片后面，直到墙为止的空间中，就会产生一个阴影体。任何东西只要伸进这个空间，就会处于阴影中。

那么，现在的问题就是，观察者在旁边盯着墙上的某一个点，要判断它到底有没有在名片的影子（阴影体）里？此时，我们只需要判断从观察者出发的视线，穿过了多少次阴影体的边界即可。比方说只穿过了一次，那么这个点就在阴影体里；但如果穿过两次，就说明视线“先进去，又出来”，这个点就不在阴影体里了。

实际实现时，阴影体常借助**模板缓存** (Stencil Buffer) 来做计数。可以把它理解为：先正常绘制场景，得到深度缓存；再绘制阴影体的正面和反面，当某个像素对应的视线进入阴影体时，模板值加一；离开时减一；最后模板值不为零的位置，就属于阴影区域。这一方法的优点是几何意义很清楚，而且由于它不是基于图像采样近似，所以理论上能够生成边界清晰、位置准确的硬阴影。它的主要问题在于性能开销较大，尤其当阴影体的面片很多、覆盖像素很多时，绘制代价会很高。

再看阴影图。它的思路和阴影体几乎相反。阴影图并不直接构造阴影区域的几何体，而是“站在光源那里看场景一次”。也就是说，把光源当成视点，用 z-buffer 渲染场景，记录从光源出发沿各个方向看到的最近深度。这样得到的一张深度图，就叫阴影图。随后，再从真正的观察视点去渲染场景。当我们算到某个可见点时，只要把它变换到光源视角下，查询它在阴影图中的对应位置，比较一下“它到光源的真实距离”和“阴影图记录的最近深度”，就能知道它是不是被别的物体挡住了。

这里的判断原则非常简单：如果当前点到光源的距离，大于阴影图中该方向记录的最近深度，那么说明在光源看来，前面已经有别的物体挡住它了，因此它位于阴影中；反之，如果两者相等或很接近，则说明它本身就是光源沿该方向看到的最前表面，不在阴影中。阴影图的最大优点是速度快、实现方便，并且非常容易与现代图形硬件结合，因此在实际系统里用得极广。它的主要缺点是属于基于图像分辨率的方法，所以会受到 shadow map 分辨率和深度精度的影响，容易出现锯齿

齿、失真、偏移等采样伪影。

把这两类方法放在一起比较，就能看出一个非常典型的图形学主题：一种方法几何上更严格、阴影更锐利，但代价偏大；另一种方法速度快、实现普遍，却会受采样分辨率影响。图形学里很多算法选择，本质上都没有绝对的先进，而要考虑谁更适合当前的任务约束。

从本章我们可以看出，真实感渲染并非一个单一的算法，而是一整套从局部反射到全局传播、从点光照到整片网格、从直接可见到阴影遮挡的体系。简单光照模型告诉我们物体表面为什么会有明暗和高光；着色技术告诉我们怎样把点上的光照推广到整个多边形网格；光线跟踪告诉我们怎样显式追踪反射和折射路径；阴影算法则告诉我们怎样把空间中的遮挡关系准确地投射到图像上。只要把这几条线索都理清，第七章看起来虽然内容多、术语杂，但其实逻辑是非常连贯的。

本节小练习

1. 简答题

为什么说环境光只能算是一种“间接照明的近似替代”，而不能真正等同于全局光照？

2. 判断题

Gouraud 着色与 Phong 着色的根本差别在于：前者插值法向量，后者插值颜色。（____）

3. 简答题

请说明镜面反射指数 n 变大时，高光区域会出现什么变化，并解释这种变化为什么对应“表面更光滑”。

4. 简答题

阴影体算法与阴影图算法都能生成阴影。请各用一句到两句话概括它们的核心思路，并分别指出它们最典型的一项优势或局限。

参考解答

1. 环境光把场景中复杂的多次反射和漫散射效应，粗略地压缩成一个各向均匀、空间中近似恒定的背景亮度项，因此它只能近似表达“不是直接照到也不会完全黑”这件事。真正的全局光照需要显式考虑光在不同物体之间的传播路径、反射与折射关系，而环境光并没有处理这些具体传播过程。
2. 错。Gouraud 着色是在顶点上先计算颜色，再对颜色做插值；Phong 着色是先对法向量做插值，再在像素级别计算颜色。
3. 当 n 变大时，镜面反射亮度会在反射方向附近衰减得更快，因此高光区域会更小、更集中、更尖锐。原因在于表面越光滑，反射越接近理想镜面，能量越集中在反射方向附近；表面越粗糙，散射越明显，高光就会更宽、更散。
4. 阴影体算法的核心是构造遮挡物对应的三维阴影体，并判断视线到达可见点前是否落在该体积内部。它的典型优势是几何意义清楚、能得到清晰准确的硬阴影，局限是体积面片多时性能压力较大。阴影图算法的核心是先从光源视角记录场景最近深度，再在正式渲染时比较当前点到光源的距离和阴影图深度。它的典型优势是实现高效、硬件支持广，局限是会受到阴影图分辨率和深度精度影响。

第八章 AI 加速的实时渲染

本节概要

这一章讨论的是一个非常现代的问题：当人们已经不满足于“能实时渲染”，而希望“既快又清晰、既稳定又真实”时，图形学和人工智能是怎样结合起来的。整章大致有三条主线。第一条是**深度学习超采样**，它回答的是：如果每一帧不能按目标分辨率完整渲染，能否借助历史帧、运动信息和神经网络，把较低成本的结果重建成更高质量的图像。第二条是**深度学习帧生成**，它关注的是：如果真正昂贵的是“完整渲染每一帧”，那么能否只渲染部分关键帧，再利用算法补出中间帧，从而提高显示帧率或降低离线渲染成本。第三条是**深度蒙特卡洛去噪**，它要解决的是：路径追踪在低采样数下为什么会有大量噪声，以及怎样利用滤波器和神经网络把这些噪声压下去。

学完这一章后，读者应当能够理解 TAA 为什么既像抗锯齿技术，又像时间维度上的超采样；知道历史重投影、历史验证和样本累积各自解决什么问题；理解神经超采样为什么要使用深度图、运动矢量和 G-buffer 之类的辅助信息；能够区分帧内插和帧外推在延迟上的本质差异；也能够说明实时路径追踪为什么必须依赖高质量去噪，以及 EAW、SVGF、KPCN 这几类方法各自的基本思路。

知识点讲解

如果把前面几章连起来看，会发现图形学一直在做一件事情：在有限计算资源下，尽量把图像做得更接近人眼所期待的效果。早期更多是改进几何表示、光照模型、反走样和消隐；到了今天，一个新的现实问题变得越来越尖锐，那就是现代场景本身已经太复杂了。分辨率越来越高，材质越来越多，阴影和反射越来越精细，显示设备又要求高帧率、低延迟和时间稳定性。在这种情况下，单纯依靠“每个像素都老老实实完整算一遍”的路线，代价往往会高得难以接受。于是，一个非常自然的想法出现了：既然很多帧之间并非毫无关系，很多高质量结果也并不是必须完全从零开始算起，那么能不能让算法主动利用已有信息，把“少算一点”和“看起来仍然很好”统一起来？这一章的内容，就是围绕这个问题展开的。

我们先从最基础的一条线索说起，也就是**从抗锯齿到超采样**。前面学反走样时，我们已经知道，锯齿和闪烁的根源在于采样不足。最直接的改进方法当然是提高采样率，但这会立刻带来更高的着色成本和显存开销。于是，在实时渲染中，时间抗锯齿 TAA (Temporal Anti-Aliasing) 逐渐成为一个非常重要的折中方案。它的核心思想不是直接在单帧内部把采样密度一下子提得很高，而是把本来应该在空间中完成的超采样，分摊到多个相邻帧里去做。也就是说，每一帧只采一点点不同的信息，再把这些信息在时间轴上累积起来，最终得到接近高采样率的效果。

要理解 TAA，先要抓住它的四个关键步骤。第一步是**抖动采样**。在渲染每一帧时，系统会对投影矩阵施加一个很小的子像素级偏移，使得相邻帧对同一物体的采样位置略有不同。第二步是**历史重投影**，也就是利用运动矢量把前一帧的结果映射回当前帧的像素位置。第三步是**历史有效性检查**，因为前一帧看到的内容并不一定仍然适用于当前帧，例如物体移动、遮挡关系变化、光照突变时，旧数据很可能已经失真。第四步是**样本累积**，把当前帧的新样本与修正后的历史结果加权混合，生成最终输出，并把它继续作为下一帧的历史输入。只要把这四步串起来，TAA 的整体结构就清楚了：先制造跨帧样本差异，再对齐，再筛查，再累积。

其中最核心的数学表达其实并不复杂。若把第 n 帧像素 p 的输出颜色记为 $f_n(p)$ ，把当前帧新得到的样本颜色记为 $s_n(p)$ ，把通过重投影算子 π 从前一帧取得的历史颜色记为 $f_{n-1}(\pi(p))$ ，那么一个典型的指数平滑累积式可以写成

$$f_n(p) = \alpha s_n(p) + (1 - \alpha) f_{n-1}(\pi(p))$$

这里， α 是混合因子。它越大，说明我们越相信当前帧的新样本，图像响应更快，但时间上的平均效果会减弱；它越小，说明我们越依赖历史数据，画面会更稳，但也更容易留下拖尾和延迟感。常用的一个经验值是 $\alpha = 0.1$ ，这意味着新样

本只占十分之一，剩下的大部分贡献来自历史累积。初看之下，这似乎有些“保守”，但正是这种保守，让 TAA 在稳定场景中能够表现出接近多样本超采样的效果。

不过，样本累积并不是简单地把过去的颜色拿过来平均一下就结束了。真正麻烦的地方在于：**历史数据未必可信**。如果一个物体刚刚移开，原本被它挡住的背景现在第一次暴露出来，那么前一帧在这个位置上的颜色很可能根本不是背景，而是前景物体留下的旧信息。如果此时还盲目混合，就会出现典型的**鬼影**。再比如一个高光快速移动，或者阴影边缘迅速变化，旧帧中的亮度分布已经和当前情况不符，这时继续累积就会产生拖尾和时间延迟。因此，现代 TAA 的重点并不是有无历史，而是“怎样判断历史能不能用，以及能用多少”。

围绕这个问题，我们来看两类经典的历史修正思想。第一类是 **AABB 裁剪**。它会查看当前帧某个像素周围一个局部邻域，例如 3×3 邻域，把这些样本颜色在颜色空间中围成一个轴对齐包围盒，然后把历史颜色钳制到这个范围内部。若历史值偏离过远，就被拉回到一个更合理的区间里。为了让这个颜色范围更紧，常会在 **YCoCg** 颜色空间中做这件事（RGB 颜色空间的一种简单无损转换形式，将颜色分离为亮度 Y 和两个色度分量：色度绿色 Cg 与色度橙色 Co）。与 RGB 相比，人类视觉系统对亮度比对颜色更敏感，因此在这种表示下，二者分离得更自然，颜色分布也更容易被包围盒约束。第二类是**方差裁剪**。它不再是简单取一个包围盒，而是先估计邻域颜色的均值 μ 和标准差 σ ，再构造一个更鲁棒的动态范围

$$[\mu - \gamma\sigma, \mu + \gamma\sigma]$$

其中， γ 控制允许历史值偏离均值的程度。这样做的好处是，当邻域中出现离群值时，算法不至于被少量异常颜色直接带偏，而能够根据整体波动程度更平滑地决定裁剪强度。

把这一层机制理解透之后，我们就能自然进入一个更进一步的问题：在前面的 TAA 介绍中，其目的就是将原本为 1080P 的一帧，变为仍然是 1080P 但没有锯齿的一帧。既然 TAA 本质上是在时间维度上做超采样，那么它是否还可以承担“**时间性超采样**”甚至“**时间性超分辨率**”的任务？换句话说，我们能不能故意让每一帧只以较低分辨率渲染，然后借助多帧的抖动采样、历史重投影和累积能力，把它重建成一个更高分辨率的输出结果？

答案当然是可以，这也是现代很多实时系统极其重要的一条路线。例如，我们让游戏引擎只去渲染一个 720P 的极小画面。这时候，720P 里面的每一个像素，由于抖动采样，它在空间中代表的位置是跨越像素格子的。当把多个残缺画面叠加到目标帧时，显卡利用这些不同位置的“新鲜像素碎片”，竟然能直接把一张 720P 的小图，拼凑、重建成了一张 1080P 的大图。

这条路线之所以成立，是因为低分辨率输入和高分辨率输出之间并非完全无关的。虽然单帧里低分辨率样本很稀疏，但如果把多帧中略有偏移的采样位置合在一起看，原本缺失的信息就可能逐步被补回来。于是，时间性超采样可以理解成一种更积极的 TAA：它不只是用时间信息来压锯齿，还主动用时间信息来“补像素”。和传统的双线性插值相比，这种方法的关键优势在于：它利用的不只是当前帧中相邻像素的空间关系，而是多帧之间的真实历史样本，所以更有机会恢复细节，而不是把图像简单拉模糊。

棋盘格渲染是其中一种很具有代表性的变体。它的做法很巧妙：不再是每一帧都随机抖动地采一点，而是用一种确定性的规则，让一帧只渲染屏幕上一半像素，排列方式像棋盘格；下一帧再去渲染另一半像素。这样做可以把单帧的着色成本几乎直接减半。随后，再利用 TAA 的重投影与累积机制，把前后两帧的信息拼成更完整的高分辨率图像。这个思路之所以重要，不仅在于它高效，更在于它说明了现代实时渲染的一个典型方向：不再坚持“单帧必须自给自足”，而是允许帧与帧之间共享信息。

当然，TAA 并不是没有代价。恰恰相反，它之所以一直是讨论热点，就是因为它带来了一整套非常独特的质量问题。第一类问题是**模糊**。由于历史颜色要经过反复重采样、重投影和加权混合，若插值和裁剪稍有不当，本来就很细的高频纹

理，例如子像素细线和远处栅栏，就很容易被抹平。第二类问题是**鬼影与时域延迟**。旧帧中的无效信息如果没有被及时排除，就会在当前帧留下拖影；而即便没有明显鬼影，快速变化的高光、阴影和反射也可能因为过度依赖历史而显得“反应偏慢”。第三类问题是**时间不稳定性**。在高频图案场景中，抖动采样会让相邻帧看到的局部颜色分布剧烈波动，这反过来又会逼迫历史修正不断重新裁剪历史值，最终造成闪烁。可以说，TAA 最大的难点一直都是如何在清晰度、稳定性和响应速度之间找到平衡。

到这里，我们就可以进入本章第一条更现代的路线，也就是**神经网络驱动的超采样**。传统 TAA 和时间性超采样已经说明，历史帧、运动矢量和低成本输入确实能帮助我们恢复高质量图像。但传统方法通常依赖人为设计的滤波器、启发式裁剪规则和若干经验参数，因此在纹理细节恢复、鬼影抑制和时间稳定性之间，很难在各种场景下都做到理想折中。于是，一个非常自然的想法出现了：既然渲染管线本身可以提供丰富的辅助信息，例如颜色、深度图、运动矢量，甚至法线与材质缓冲，那么能否让一个神经网络自己去学习“怎样把这些信息组合起来，恢复出更高质量的输出”？

这种思路的代表之一是 **NSRR** (Neural Supersampling for Real-time Rendering)。它是一种相对通用的神经超采样框架。其基本目标非常明确：给定低分辨率、多帧、带辅助信息的渲染结果，输出更高分辨率、更稳定、更少伪影的图像。若把它拆开看，结构并不神秘，通常可以概括为四个模块：**特征提取**、**时间重投影**、**特征重加权**、**图像重建**。特征提取阶段先对颜色和深度做卷积，得到适合网络处理的表示；时间重投影阶段利用亚像素精度的运动矢量，把历史特征拉回当前帧坐标；特征重加权阶段再决定哪些历史特征更可靠、哪些应该减弱，这也是取代了传统 TAA 容易产生鬼影的关键一步；最后图像重建阶段把这些信息融合成当前帧的高分辨率输出。

我们知道，在计算机视觉和深度学习中，卷积不仅仅是比较相似度，更深的本质是一个特征提取器。一个卷积核在图像上滑过，就像是一个具有特定功能的滤镜。它可以提取出边缘、提取出细节、或者提取出高频噪点。因此，他就能提炼出比颜色更丰富的、更多的信息，帮助 NSRR 得到更高质量的图像。与传统 TAA 相比，其最大的不同就在于每一步中的“权重怎么取、信息怎么融合”，不再完全依赖手工规则，而是通过训练学出来。

不过，即使使用神经网络，这里有一个事实也没有改变，那就是：**辅助信息依然至关重要**。深度图告诉网络几何层次和遮挡边界在哪里，运动矢量告诉网络历史信息怎样对齐，颜色信息提供直接观测，而后续可能还会有法线、反照率等额外缓冲帮助网络理解场景结构。也就是说，在神经超采样中，我们并不能说“只要有网络，别的都不重要”，恰恰相反，它是建立在图形管线已经能提供大量结构化信息的基础上的。图形学在这里没有被 AI 替代，而是为 AI 提供了高质量的中间表示。

说完超采样之后，我们进入第二条主线，也就是**深度学习帧生成**。这个问题和超采样非常像，只不过这次缺少的不再是空间分辨率，而是时间帧率。设想一个场景：如果真正昂贵的是完整渲染一帧，那么是否可以只渲染关键帧，再在它们之间生成一张或多张中间帧，从而让显示端看起来帧率更高？这就是视频插帧和渲染插帧要解决的问题。它的基本单位不再是“单帧里的像素细节恢复”，而是“相邻帧之间的运动与内容重建”。

传统视频插帧的核心概念是**光流**。光流描述的是像素在相邻帧之间的表观运动场。若我们知道了前后两帧之间每个像素大致是怎样移动的，就可以把一帧中的内容扭曲到中间时刻，再与另一帧对应结果融合，从而合成目标中间帧。把这个过程分解来看，通常有三步：第一步是**运动估计**，计算输入帧之间的光流；第二步是**运动补偿**，也就是根据光流把已有像素搬运到中间位置；第三步是**图像融合**，处理两个方向搬运后的结果，特别是遮挡边界和冲突区域。这套思路很自然，因为它本质上就是先回答“东西往哪儿动”，再回答“中间那一刻应该长什么样”。

在这里，我们主要看两个代表模型。第一个是 **RIFE**，它的重点非常鲜明，就是强调**实时性**。很多视频插帧方法在质量上不错，但速度太慢，而 RIFE 的价值就在于尽可能把“能实时跑”这件事做出来。第二个是 **IFRNet**。它体现了一种更精细的思想：传统插帧往往是先完全确定运动，再根据运动去合成内容，这种严格串行的路线有时不够理想，因为内容重建质量本身也会反过来影响运动判断。于是 IFRNet 试图在一个统一框架里，让**中间流场估计**和**中间特征重建**互相促进。更准确的流场可以帮助特征对齐，而更好的中间特征又能为流场估计提供更完整的上下文和遮挡线索。这种“联合优

化”的思想，是现代很多深度模型优于传统分步方法的重要原因。

如果把帧生成问题搬到渲染领域，事情会进一步变得有意思。因为渲染场景和普通自然视频最大的不同在于：它不只有最终颜色帧，还能比较廉价地输出大量**辅助特征缓冲区**，也就是人们常说的 **G-buffer**。

在纯视频插帧中，我们面对的是一部电影，想把它帧数翻倍，此时 AI 只能硬着头皮去猜演员的手是怎么动的。然而，渲染插帧的场景则是我们在玩游戏，但游戏引擎是显卡自己运行的。显卡不仅知道前后两帧的画面颜色，还知道物体离屏幕多远、法线图中间产物。它们就是 G-buffer，都可以在渲染管线中相对直接地得到。其价值非常大，因为颜色本身往往会受光照、阴影、纹理和反射影响而变化复杂，但几何和材质缓冲则更稳定，也更有利于判断真实运动和遮挡边界。因此，针对渲染的插帧方法往往会把关键帧的颜色和这些辅助特征一起送入网络，利用它们来提高运动估计精度，并改善中间帧的遮挡处理。

沿着这一思路，我们会面临两个很实际的问题。第一，是**非线性直接合成的风险**。如果让网络直接输出最终像素颜色，虽然看起来端到端很方便，但它有时会带来色彩伪影，而且在 Alpha 通道、多层光照或多缓冲一致性要求较高的场景中，也不一定稳定。第二，是**全局插值的效率问题**。并不是整幅图像的每个区域都同样难插值。很多静态背景区域几乎不需要复杂处理，而真正棘手的往往是少数快速运动、遮挡变化或高频着色区域。如果系统必须等待未来帧到达并完成庞大的时序匹配计算，导致延迟极高，就会无法满足实时渲染的交互需求。因此，好的方法既要考虑质量，也要考虑把算力重点放在真正困难的部分。

这就把我们带到了**从内插到外推**的问题上。表面上看，帧内插和帧外推都在“凭空造新帧”，但它们在系统性质上有一个极其关键的区别：**延迟**。帧内插需要同时看到历史帧和未来帧，才能在二者之间生成中间帧。这样做的好处是质量通常更稳定，因为信息来自两侧；但代价是，系统必须等待未来帧先被渲染出来，于是凭空增加了一个额外等待过程。对于离线视频处理，这个代价可能还能接受；但对实时交互式渲染，尤其是延迟敏感系统，这就很麻烦了。相对地，帧外推只依赖历史帧来预测未来帧，不需要等待尚未到来的真实图像，因此更有希望维持较低延迟。但它的难点也更明显，因为“未来会发生什么”本来就比“前后之间补一个中间状态”更难预测。

ExtraNet 就代表了这种偏向低延迟的帧外推路线。它要解决的最大难题，是**新出现区域**的问题。若一个区域在历史帧中根本没被看到，那么单靠过去的颜色显然无法直接知道它应该长什么样。ExtraNet 的一个重要思路，是继续利用成本远低于完整着色图像的未来帧 G-buffer，例如被外推帧的深度和法线，给网络提供几何结构引导。于是，哪怕未来帧的最终颜色还没被完整算出来，网络也已经知道那一帧大致是什么几何布局。再加上历史编码器对多帧动态阴影、高光和反射变化的分析，网络就可以把任务拆成两部分：对缺少历史对应的新区域做“辐照度修复”，对能找到历史对应的位置做“无鬼影着色预测”。这说明帧外推虽然难，但并不是完全凭空猜测，而是在利用便宜而结构化的信息降低预测不确定性。

再往前一步，我们来关注 **ExtraSS** 这类把**空间超分辨率**和**帧外推**合并起来的统一框架。它的吸引力非常强，因为现实系统往往同时缺两样东西：一是单帧的空间分辨率不够，二是时间帧率不够。如果能把这两种恢复任务合并，就有机会用更少的真实渲染帧，得到更高分辨率且更高显示帧率的输出。核心思路是交替生成两类帧：一类偏重空间超分，另一类同时结合帧外推与超分。为了改善外推初值，它还会利用 G-buffer 和双边滤波从历史中采样像素，尽量先得到一个鬼影更少的初始结果；随后，再通过专门的着色精炼网络修复由 G-buffer 扭曲无法直接处理的动态阴影和反射。

从中我们也可以看出，现代实时渲染中的很多 AI 方法已经不再只解决单一问题，而是在尝试把超分、抗锯齿、帧生成、着色修补等任务放到一个统一流水线里看待。

讲完超采样和帧生成，本章最后一条主线是**深度蒙特卡洛去噪**。要理解这一点，必须回到路径追踪本身。前面在真实感渲染里已经知道，路径追踪是一种基于蒙特卡洛思想（用随机抽样的平均值来近似复杂的积分结果）的全局光照方法，即对该像素生成的多条随机路径的颜色求平均值，近似得出该像素的精确色彩。它的优点是物理意义清楚、真实感强，但它有一个非常著名的代价：**噪声**。原因也不难理解。路径追踪要靠随机采样去估计光在场景中的复杂传播，而随机采样在样本数少时总会有明显方差。若每个像素有成百上千条路径，估计自然会逐渐稳定；可如果一个实时系统每个像素只能承受 1

到几个样本，也就是低 **SPP** (Samples Per Pixel) 情况，那么图像里就会充满颗粒状噪声。这种噪声正是因为统计估计本身还没收敛。

于是，实时路径追踪几乎必然要配合**滤波器**或**降噪器**使用。它们的任务是尽量在移除随机噪声的同时，保住真正重要的结构，例如几何边缘、纹理细节和阴影边界。这是一件比看上去困难得多的事情。因为噪声和细节在局部上都可能表现为“变化很快”，如果处理不好，算法就会把真实细节也当作噪声一起抹掉。因此，去噪问题的本质并不是“怎样更强地平滑”，而是“怎样区分哪些高频变化是噪声，哪些高频变化是场景结构”。

首先，我们来看偏传统路线的 **EAW**，也就是边缘规避型小波变换滤波器。它属于**纯空间域、单帧去噪**的方法。其基本思想是在多个尺度上对图像做平滑，但平滑时并非一视同仁，而是利用法线、世界坐标等辅助缓冲区去判断边缘在哪里，从而在边缘附近减少跨边界混合。这种方法的优点是高效、直接，而且不需要等待时间信息，非常适合单帧内快速压噪。它的局限也同样明显：因为它只看当前一帧，所以很难保证动画序列在时间上的稳定性。换句话说，它可能让每一帧看起来都还不错，但连续播放时仍会闪。

正因为如此，我们引入 **SVGF**，也就是**时空方差引导滤波器**。这个方法的重要性在于，它把“时间维度”正式纳入去噪流程。它的想法和 **TAA** 有相通之处：既然前一帧的渲染结果已经包含了有价值的信息，那么我们为什么不利用它来提高当前帧的等效采样数？**SVGF** 的典型流程可以概括成两步。第一步是**时间累积**。借助 **G-buffer** 中的运动矢量，把前一帧的颜色和统计量重投影到当前帧；若重投影后发现几何上一致，说明它们来自同一表面，就可以把历史和当前的低样本结果做指数移动平均。这里的指数说的正是前面的权重系数 α 。第二步是**方差引导的空间滤波**。在时间维度累积一阶矩和二阶矩之后，我们就能实时估计每个像素亮度方差。方差越高，说明噪声越大，对应区域就需要更强的平滑；方差较低处则可以更克制地处理，从而尽量保留细节。这样一来，**SVGF** 就把“跨帧积累更多有效样本”和“在空间上按噪声程度自适应平滑”结合了起来。

走到这一步后，神经网络进入去噪领域也就顺理成章了。传统降噪器往往需要人手工设计滤波结构并调整参数，但不同场景下噪声分布、材质纹理和光照条件差异很大，固定规则很难总是奏效。于是，现代方法开始让网络直接学习从带噪输入及其辅助特征到干净参考图像之间的映射。这里有两类思路特别值得注意。第一类并不让网络直接输出最终图像，而是让网络去**学习滤波器参数**。换句话说，网络仍然服务于传统滤波器，只不过把最难调的参数交给学习过程来确定。第二类则更激进，让网络直接预测每个像素专属的滤波核，这就是 **KPCN** 一类**核预测网络**的思想。对于每个像素，网络不再共享一个统一卷积核，而是为它量身生成一个局部滤波核，于是不同位置就可以根据各自的噪声和结构特点采用不同的平滑方式。这种做法之所以强大，正是因为它兼顾了局部自适应性与卷积形式的可解释性。

不过，单帧神经去噪同样会遇到一个和传统方法类似的问题：**时间稳定性**。如果每一帧都独立去噪，即便单帧看起来很干净，连续播放时仍可能出现闪烁。因此，最后我们再将时间信息重新带回来，介绍一种将**循环神经网络**与 **U-Net** 结构结合的思路。其关键就在于“网络开始具有记忆”。当前帧的输出不再只依赖当前输入，还依赖前一帧传来的隐状态。这样，网络就能在多帧之间学习关联，理解哪些变化是真实运动，哪些只是随机噪声，从而在去噪的同时维持更平滑、更连贯的时间表现。

还有一个值得额外记住的方向：**去噪不再总是孤立存在**。当现代渲染流水线同时面对低分辨率、低帧率和低采样数三个问题时，去噪往往会与超分辨率、时间重建、帧生成等任务相互耦合。原因很简单：这些任务都在试图用有限信息恢复一个更高质量的结果。路径追踪图像去噪后再做超分，和先做时域重建再做降噪，可能会得到完全不同的效果；而在统一框架里协同处理这些问题，又可能进一步提高整体效率和视觉一致性。虽然这一方向本身还在不断发展，但读者至少应该建立这样一种认识：现代 **AI** 的加速渲染不是一个单点技巧，而是一整套“利用结构化辅助信息，把空间、时间与采样稀缺问题放在一起考虑”的方法论。

本章的逻辑实际上非常统一。**TAA** 和**时间超采样**解决的是“当单帧按目标质量完整渲染代价过高时，如何利用多帧补足空间质量”；**神经超采样**解决的是“怎样更聪明地整合颜色、深度和运动信息”；**帧生成**解决的是“怎样少渲染一些帧却维

持更高显示帧率”；去噪解决的是“怎样在样本数不够时，仍然得到稳定而清晰的路径追踪结果”。它们表面上分别对应分辨率、帧率和噪声，实质上都在回答同一个问题：当完整计算成本过高时，怎样尽量不浪费历史信息、辅助缓冲和统计结构，把已有数据用到极致。这正是 AI 加速实时渲染最核心的思想。

本节小练习

1. 简答题

为什么说 TAA 本质上可以被看成“时间维度上的超采样”，而不只是普通意义上的抗锯齿？

2. 判断题

帧内插和帧外推都能生成新帧，因此它们在输入延迟上的影响本质相同。（_____）

3. 简答题

请说明 G-buffer 在神经超采样、针对渲染的插帧和实时去噪中为什么都很重要。

4. 简答题

SVGF 相比只使用单帧空间滤波的方法，核心多做了哪两件事情？它们分别解决什么问题？

参考解答

1. 因为 TAA 的关键不在于单帧内部把采样数直接提上去，而在于通过抖动采样让相邻帧获得略有差异的样本，再通过重投影与累积把这些样本在时间轴上合并起来。这样得到的效果，本质上是在用多帧共同完成原本需要更高空间采样率才能实现的重建任务，所以它不仅是抗锯齿，也是一种时间维度上的分摊超采样。
2. 错。帧内插需要同时依赖历史帧和未来帧，因此往往必须等待未来帧先出现，这会带来额外延迟；帧外推只利用历史信息预测未来帧，不需要等待真正的未来帧，因此更有利于控制输入延迟。二者虽然都在生成新帧，但对系统时延的影响并不相同。
3. 因为 G-buffer 提供了颜色之外的更稳定、更结构化的几何与材质信息，例如深度、法线和反照率。神经超采样需要它们来帮助网络理解哪些历史信息可以安全重用；针对渲染的插帧需要它们来改善运动估计与遮挡处理；实时去噪则利用它们识别真实边缘和表面一致性，避免把不同物体或不同材质的区域错误混合。
4. 第一件事是时间累积，也就是把前一帧经过重投影后的结果与当前帧样本结合起来，提高当前帧的等效采样数，从而减轻低样本噪声。第二件事是方差引导的空间滤波，也就是利用一阶矩和二阶矩估计亮度方差，再根据方差大小动态调整平滑强度。前者主要解决“单帧样本太少”的问题，后者主要解决“不同区域噪声强弱不同，不能一刀切平滑”的问题。

第九章 传统交互技术

本节概要

这一章讨论的重点，不再是“怎样把图像算出来”，而是“怎样让人自然、稳定、高效地与图形系统发生联系”。当计算机图形学走出静态绘图，进入建模、动画、游戏、虚拟现实与增强现实之后，交互就不再只是一个附属问题，而是直接决定系统是否可用的核心因素。本章将依次介绍人机交互的基本发展脉络与经典模型，再进一步讨论虚拟现实与增强现实中的硬件、渲染、配准和交互方式，最后说明它们在图形学中的典型应用与实际约束。

知识点讲解

我们先从最一般的人机交互说起。所谓**人机交互** (Human-Computer Interaction, HCI), 就是研究人与计算机系统之间怎样交换信息、怎样发出控制、以及系统怎样反馈结果的一门技术与方法体系。值得注意的是, 这里我们不再单纯关注**图形用户界面** (Graphical User Interface, GUI), 也就是平时我们所说的“在电脑上画出桌面、用鼠标点击图标”等功能。人机交互研究的是一个更广阔而先进的领域: 人与计算机如何更好地沟通。它不仅关注“界面好不好看、好不好点”, 还研究语音控制、体感交互 (如 VR/AR)、甚至脑机接口。

一个图形系统即使有很强的几何处理和渲染能力, 如果用户无法方便地下达指令、理解反馈并修正操作, 那么这个系统就很难真正发挥价值。这也正是人机交互的难点所在。试想我们如果要实现一个三维游戏, 能让用户能够戴上 VR 头盔, 用控制器操作逼真的游戏界面, 还要能便捷地转动视角、跑动跳跃、与系统交互, 那这绝非一件易事。

从发展脉络看, 早期交互主要依赖**命令行界面** (Command Line Interface, CLI)。这种方式的优点是精确、直接, 但学习成本较高, 用户必须记忆命令名称、参数格式和执行规则, 而且缺乏直观视觉反馈。后来就出现了我们熟知的图形用户界面 GUI, 此时用户不再只面对一串文本命令, 而是通过窗口、图标、菜单和指针与系统交互。再往后, 个人计算机普及推动了 **WIMP** 范式的发展, 也就是 **Windows、Icons、Menus、Pointer** (窗口、图标、菜单、指针)。其允许用户直观识别与操作、同时运行和查看多个应用程序或文档, 也正是现代个人电脑操作系统的基石。这种范式之所以影响深远, 是因为它把“操作”和“看到结果”更紧密地联系起来, 用户点击一个图标、拖动一个对象、打开一个窗口, 都能立刻得到相应反馈。很多常见图形软件, 例如图像编辑软件和三维建模软件, 本质上都建立在这一思路之上。

不过, 传统的键盘与鼠标并不是交互演化的终点。随着传感器、语音识别、手势识别、触觉反馈和眼动跟踪的发展, 现代交互逐渐走向**多模态交互**。所谓多模态, 就是系统不再只理解一种输入, 而是同时利用语音、动作、视线、控制器乃至触觉反馈来建立更自然的交互回路。进一步发展之后, 便形成了沉浸式交互环境, 这也就是虚拟现实与增强现实得以兴起的重要基础。

理解交互系统时, 最重要的是交互过程本身。最基本的描述方式是**交互循环模型**。在这个模型中, 用户先有意图, 再执行动作; 系统对动作进行处理并给出反馈; 用户感知反馈、解释结果, 再决定下一步行动。这个过程可以简要概括为: 用户意图 → 动作执行 → 系统处理与反馈 → 用户感知与评价。之所以称为“循环”, 就是因为交互不是一次性完成的, 而是不断根据反馈修正目标的迭代过程。

在更细致的层面上, 常用的是 **Norman 七阶段模型**。它把一次完整交互分为七个步骤: 建立目标、制定意图、指定动作、执行动作、感知系统状态、解释系统状态、评估结果是否符合目标。这个模型中有两个很经典的概念。第一个是**执行鸿沟**, 它指用户明明知道自己想做什么, 却不容易把目标转化成合适操作; 第二个是**评价鸿沟**, 它指系统明明已经给出结果, 但用户不容易理解结果意味着什么。许多优秀界面的设计, 实质上都在努力缩小这两道鸿沟。

图形学系统对交互还有两项特别敏感的要求: **实时性和可控性**。实时性是说系统必须尽快响应用户操作, 尤其在游戏、建模和沉浸式场景里, 较明显的延迟会破坏使用体验, 严重时甚至带来不适。可控性则强调操作应当稳定、直观、可预测, 用户应能大致预判自己的动作会引起什么后果。如果系统响应过于复杂、反馈不稳定或者结果不可预测, 那么即使图像质量很高, 交互体验仍会很差。图形系统的设计, 很多时候就在这两者之间寻找平衡。

从设备角度看, 输入设备的任务是把人的意图转换为机器能够处理的信号。最传统的输入设备是键盘、鼠标和数位板。键盘擅长输入命令、文本和精确控制, 但缺乏空间直观性; 鼠标通过二维平面上的移动控制屏幕指针, 适合桌面图形界面, 却天然受限于二维输入; 数位板配合手写笔可以提供高精度坐标输入和压力感应, 因此在数字绘画、纹理绘制和自由曲线造型中非常常见。到了三维交互和沉浸式交互阶段, 还会进一步用到运动捕捉、手势识别和手部跟踪技术。它们依赖摄像头、红外设备、惯性传感器或深度相机来实时获取人体动作轨迹, 使用户可以通过身体动作直接操控虚拟对象。

与输入相对, 输出设备负责把系统的计算结果以可感知的方式呈现给用户。最常见的是视觉显示设备。普通显示器通过

像素阵列呈现图像，人们通常会关心分辨率、刷新率、色彩还原和动态范围等指标。常见显示技术包括 LCD、OLED 与 Micro-LED。除了单一显示器之外，还可以通过多显示系统构成超宽屏、环绕屏甚至球幕，以扩展视野并提升沉浸感。再往前一步，就会涉及**立体显示与全息显示**。立体显示利用双眼视差，为左右眼提供略有差异的图像，从而形成深度感；常见方式包括偏振光眼镜、主动快门眼镜和裸眼 3D。全息显示则基于光的干涉与衍射，希望更直接地重建空间中的三维光场。

视觉并不是唯一输出通道。交互系统还经常使用**触觉反馈和力反馈**。触觉反馈通过振动、摩擦或压力模拟接触感觉，让用户感知表面纹理、粗糙程度或接触事件；力反馈则更进一步，通过机械阻力或外力作用，让用户感受到重量、硬度和惯性等物理属性。二者在虚拟装配、训练模拟和沉浸式系统中都很重要。若再结合声音，就会形成更加完整的多感官反馈环境。尤其在虚拟现实系统中，声音不仅要“能听见”，还应具备**空间性、同步性和低延迟**。换句话说，声源方向应尽量与虚拟场景中的位置关系一致，用户转头后听到的声音方向也应及时更新，并且声音与画面反馈不能明显错位，否则沉浸感会迅速下降。

交互技术在图形学中的应用非常广泛。以三维建模为例，早期建模往往更依赖命令或脚本，而交互式编辑允许用户直接拖拽、旋转、缩放和即时预览，这大大降低了操作门槛。在动画和游戏中，交互技术则决定了人物控制是否流畅、反馈是否自然以及沉浸感是否足够。也就是说，交互并不只是一个“界面问题”，它已经深度嵌入图形学系统的性能、算法和体验设计之中。

在此基础上，我们进入第一个更具体的主题：**虚拟现实** (Virtual Reality, VR)。虚拟现实可以理解作为一种由计算机生成三维沉浸式环境、并通过多感官交互让用户获得身临其境体验的技术形态。它的发展可以追溯到二十世纪六十年代。一个常被提到的早期系统是 Morton Heilig 在 1962 年提出的 **Sensorama**，它尝试将立体视觉、声音乃至气味结合起来，构造初步的沉浸体验。

虚拟现实通常强调三个特征。第一是**沉浸感**。用户不再只是看着屏幕上的场景，而是仿佛进入场景内部，这依赖宽视角显示、立体视觉、空间音频和触觉反馈等多种技术共同支持。第二是**交互性**。用户的动作、手势或控制器输入应能实时改变虚拟环境，系统也要及时反馈结果。第三是**想象性**。虚拟现实不只是复制现实，还可以创造现实中并不存在但在交互上又自洽的世界。这样一来，VR 就和传统桌面交互形成了鲜明对比：传统方式往往是用户面对二维屏幕，像旁观者一样间接操控图形系统；而 VR 更倾向于让用户直接处在虚拟环境之中，以身体动作为媒介与对象发生联系。

从系统构成上看，VR 往往可以分成硬件层、软件层，以及系统集成与实时控制层。硬件层包括输入与输出设备。典型输入设备有头戴式显示器中的姿态传感器、体感控制器、数据手套、眼动追踪器和动作捕捉系统等；输出设备则包括双目立体显示、空间音频，以及触觉和力反馈装置。软件层主要由渲染引擎与交互引擎组成。前者负责把三维场景实时绘制出来并输出给显示设备，后者负责管理输入、处理事件、更新场景状态，把用户意图映射成虚拟环境中的动作与反馈。系统集成层则要保证这些部分能稳定协同工作，确保从传感器到渲染再到显示的整个数据流及时同步。

正因为如此，VR 对实时性的要求比普通桌面图形系统更严格。工程上常希望端到端延迟控制在约 20ms 或更低，同时保持 90Hz 甚至更高的刷新率。如果延迟过高，用户头部和身体已经发生运动，但画面还没有及时更新，就会出现视觉信息与前庭感觉之间的不一致，从而引发眩晕、恶心或空间错位感。这也是所谓“模拟器眩晕”的重要来源之一。由此可见，VR 的实时性与人体感知机制直接相关。

在图形渲染方面，VR 的一个显著特点是**双目渲染**。人眼依靠左右眼之间的双眼视差感知深度，因此 VR 系统需要在同一时刻分别为左眼和右眼生成略有差异的图像。这样虽然能够形成强烈的立体感，但代价也很明显：渲染负担几乎会成倍增长。为了缓解这一压力，现代系统常会使用**注视点渲染** (Foveated Rendering) 一类优化技术。它依据眼动追踪结果判断用户当前的注视中心，在中心区域使用较高分辨率渲染，而在周边视野适当降低分辨率。这样做利用了人眼在中央视觉区域更敏感、在周边区域分辨率要求较低的特点，能够显著减少像素着色负担。

虚拟现实在图形学中的应用十分广泛。它可以用于三维建模与虚拟场景设计，让用户直接在三维空间中操控对象；可以

用于沉浸式游戏与动画体验，通过头部追踪、手势识别和空间音频构造强交互场景；还可以用于科研与教育，例如虚拟实验室、飞行模拟和危险环境训练等。这些应用共同说明，VR 的价值不仅在于“看上去很真实”，更在于它改变了人与图形系统互动的方式。

与虚拟现实相对应的，是**增强现实** (Augmented Reality, AR)。增强现实并不试图让用户完全脱离真实环境，而是在真实世界的基础上叠加虚拟信息，使二者在同一视图中融合。它的核心特征通常概括为三点：第一是**虚实融合**，也就是真实环境和虚拟元素同时出现在用户视野中；第二是**实时交互**，用户可以用手势、语音、视线等方式即时操纵虚拟内容；第三是**三维配准**，虚拟对象必须在位置、姿态和尺度上与真实环境正确对齐，否则增强效果就会立刻失真。

因此，AR 与 VR 虽然都属于沉浸式交互技术，但它们的系统目标并不相同。VR 倾向于让用户进入一个完全虚拟的世界，强调封闭环境中的沉浸；AR 则保留真实环境，把虚拟对象作为附加信息层叠加其上，更强调虚实一致性和场景对齐。也正因为如此，AR 对配准、环境感知和真实世界光照匹配的要求往往更加突出。

增强现实的技术基础，可以从感知采集、显示呈现和空间理解三个方面来理解。落脚到具体技术上，分别对应的就是传感器与摄像头技术、显示技术与定位配准。我们依次来看。

首先是传感器与摄像头技术。AR 设备往往配备多种传感器，用于估计设备运动并感知环境。一个非常常见的组合是 **IMU**，即惯性测量单元 (Inertial Measurement Unit)，其中通常集成加速度计、陀螺仪和磁力计。加速度计测量加速度变化，能够反映平移相关信息；陀螺仪测量角速度，用于估计旋转；磁力计则提供地磁参考，有助于姿态校正。IMU 的优点是响应快，适合捕捉高频运动，但它容易产生累积误差，因此通常要与视觉信息结合使用。

其次是显示技术。AR 的主流显示方式可以分成三类。第一类是**光学透视显示**，它通过半透明镜片或光波导把虚拟图像叠加到用户对真实世界的直接观察上，优点是保留自然视觉，但容易受到环境光影响。第二类是**视频透视显示**，系统先用摄像头采集真实场景图像，再在图像上叠加虚拟内容并显示给用户，这种方式更容易进行复杂图像处理 and 像素级融合。第三类是**全息显示**，它希望用干涉和衍射等方式更真实地重现空间光场，不过实际实现难度也更高。

再次是定位与配准，也就是让虚拟对象稳定地落在真实世界“该在的位置上”。如果在桌上放了一杯虚拟的奶茶，当你转头时，这杯奶茶必须死死“粘”在桌面上，不能跟着眼睛一起飘走。这就需要机器去“理解”空间。这里要特别区分**定位**与**配准**。定位更强调设备或用户自身在空间中的位置与姿态估计；配准则更强调把虚拟对象在三维空间中对齐到现实场景的对应位置、方向和尺度。常见配准方法包括**基于标记的配准**和**无标记配准**。基于标记的方法使用二维码、特定图案或预定义图像作为参考点，系统识别标记后就能确定虚拟对象的摆放位置。无标记方法则不依赖专门标记，而是利用自然特征点跟踪、环境几何恢复以及 **SLAM** (Simultaneous Localization and Mapping, 同时定位与建图) 等技术完成定位与配准。后者更灵活，也更接近现代移动 AR 系统的主流方案。

在交互方式上，AR 通常比传统桌面交互更强调自然性，也比 VR 更强调与真实环境的协调。常见方式包括手势交互、语音交互、眼动交互和多模态协同。手势交互允许用户直接通过身体动作操控虚拟信息，不必依赖额外控制器；语音交互适合菜单导航、任务切换和信息检索，效率高但容易受噪声影响；眼动交互通过检测用户关注点来降低手部负担；多模态交互则试图把多种输入同时融合，使系统更像在理解“人的行为整体”，而非只识别某一种单独命令。

增强现实在图形学中的应用，同样很有代表性。第一类是三维模型叠加与实时渲染，例如虚拟家具摆放。此时系统不仅要吧模型画出来，还要让它在尺度、位置和阴影上看起来真的处于房间中，因此会涉及实时光照估计、遮挡关系判断和环境映射。第二类是工业与设计辅助，例如在真实装配场景中叠加线路走向、部件位置或维修提示，让工人在真实环境里边操作边查看信息。第三类是医学教育与手术导航，在医生视野中叠加 CT 或 MRI 对应的三维结构，使关键器官与病灶信息能与真实人体位置对齐。

最后，也要看到 AR 面临的一些系统约束。**延迟**会直接破坏虚实对齐效果，甚至导致操作失败；**功耗**较高，因为系统往往要同时运行传感器采集、定位、渲染和通信；**视场角**也是常见瓶颈，许多 AR 设备的可显示区域仍明显小于自然视野

范围，因此虚拟内容只能出现在视野的一部分。可见，AR 的难点并不只是“把东西叠上去”，而是要在有限设备条件下实现自然、稳定、低延迟的虚实融合。

回顾本章，主线是非常清晰的：传统交互技术先回答“人怎样通过设备与系统交换信息”，而 VR 与 AR 则把这个问题进一步推进到沉浸式空间环境之中。前者强调命令、界面、反馈与控制逻辑，后者强调传感器、配准、低延迟渲染与多模态交互。无论形式如何变化，交互系统真正追求的目标始终一致：让人的意图更容易被表达，让系统的状态更容易被理解，并让反馈尽可能自然、及时而可信。

本节小练习

1. 简答题

为什么说 GUI 与 WIMP 范式的出现，使图形系统的学习成本和操作方式都发生了明显变化？

2. 判断题

增强现实和虚拟现实都属于沉浸式交互技术，因此它们的核心任务完全相同。（_____）

3. 简答题

为什么 VR 系统对低延迟、高刷新率和声音同步都有较高要求？

参考解答

1. 因为 GUI 与 WIMP 把很多原本需要记忆文本命令的操作，转化成了窗口、图标、菜单和指针上的直观动作。用户不必先掌握复杂命令语法，就能通过点击、拖动、选择等方式完成任务，并且能立即看到视觉反馈。这样既降低了学习门槛，也让操作结果更容易被理解和修正。
2. 错。二者虽然都强调沉浸式交互，但 VR 主要是构造一个完整虚拟环境，让用户进入其中；AR 则是在真实环境基础上叠加虚拟信息，更强调虚实融合与三维配准。因此二者的系统目标并不完全相同。
3. 因为 VR 中用户的头部、身体和视线会不断运动，系统必须快速更新左右眼图像、控制空间音频方向并保持反馈同步。如果延迟过高，或者画面与声音更新不同步，就会造成感知错位，破坏沉浸感，严重时还会引发眩晕、恶心和空间错位感。

第十章 真实图形学应用

本节概要

前面几章主要讨论的是图形学的基本原理、建模方法、渲染技术和交互方式，而这一章要回答的是另一个很实际的问题：这些方法最终到底用在什么地方。围绕这一点，可以看到两条不同但彼此联系的主线。一条是更强调**客观真实**的应用，例如医疗健康，它要求尽可能准确地表达真实结构和真实关系；另一条是更强调**主观真实**的应用，例如工业设计和影视制作，它更看重设计表达、视觉说服力和交互体验。本章将沿着这两条线展开，分别介绍医疗健康、CAD 与工业设计、影视制作中的代表性应用。

知识点讲解

如果把“真实”这个词仔细拆开来，就会发现它并不总是同一种含义。有些场景里，真实意味着结果必须尽量忠实于客观世界本身，哪怕只是毫米级的误差都可能带来严重后果；另一些场景里，真实则更偏向于“看起来可信”“表达上成立”

或者“足够符合设计意图”。前一种可以称为**客观真实**，后一种则更接近**主观真实**。计算机图形学之所以应用极广，正是因为它既能服务于前者，也能服务于后者。

首先来看客观真实的代表领域，也就是**医疗健康**。在医疗场景里，图形学的任务远不只是把图像显示得更好看，而是要帮助医生理解真实的人体结构、病变位置及其与周围组织之间的关系。这里涉及的关键工作包括医学影像数据的处理、三维重建、分割和可视化。换句话说，系统需要把原本分散在一层层切片中的信息重新组织起来，使医生能够从空间角度理解病灶与器官的整体关系。

一个最基础也最重要的步骤，是把二维医学图像转化为三维结构。常见数据来源有 **CT** (Computed Tomography, 计算机断层扫描) 和 **MRI** (Magnetic Resonance Imaging, 磁共振成像)。二者虽然都能提供人体内部信息，但适用重点并不相同。CT 依赖 X 射线成像，对骨骼、肺部和血管等结构的观察往往较为直接；MRI 则依赖磁场和无线电波，更擅长呈现软组织细节，因此在大脑、脊柱等部位检查中非常重要。图形学在这里的作用，是把这些切片式数据组织成可旋转、可观察、可测量的三维模型。这样一来，医生就不必只在一张张平面图之间来回切换，而可以更清楚地看到病变究竟处在什么位置、与哪些组织靠得近、以及手术路径应如何规划。

例如，对心脏 CT 数据进行三维重建后，系统可以清晰地展示心脏各个结构以及血管内部的狭窄或硬化情况。这类结果的价值，不在于视觉上是否炫目，而在于它能提高诊断和分析的精度。也正因如此，医疗场景中的图形学应用特别重视几何准确性、结构清晰度和交互可解释性。

除了重建之外，另一个非常关键的任务是**医学图像分割**。所谓分割，就是从复杂影像中把真正关心的区域提取出来，例如肿瘤、器官、血管或者病灶边界。没有分割时，一整幅三维影像往往包含过多无关信息，医生需要花大量精力去识别目标结构；而分割以后，感兴趣区域就能被单独高亮、建模或定量分析。近年来，这一方向与人工智能结合得非常紧密。**VISTA3D** 就是一个面向三维医学图像分割的基础模型，它的目标正是从复杂三维数据中提取关键区域。在三维医学场景里，轴向、矢状和冠状视图往往要同时结合使用，因此一个好的分割模型不仅要在单个切面上表现良好，还要保证不同视角之间具有一致性。

与此类似，**MedSAM** 则可以理解作为一种更通用的医学图像分割基础模型。它面向多种影像模态，在大规模图像与掩模配对数据上训练，希望获得比传统“只针对某一种模态单独训练”的模型更强的泛化能力。这里值得特别注意的一点是：图形学和医学 AI 的结合，是在利用图形学长期积累的可视化、重建和交互能力，把 AI 给出的结果变成真正可观察、可分析、可验证的信息，而不是简单地把算法硬塞进医院流程。对于医疗场景来说，这一点尤其重要，因为一个分割结果如果无法被清楚显示和理解，它的实用价值就会大打折扣。

在重建和分割基础上，图形学还进一步进入了**虚拟手术、增强现实导航和个性化医疗**。我们依次来看这三点应用：

1. 虚拟手术系统利用三维建模、仿真与 VR 技术，为医生提供术前训练和路径演练环境。与传统二维阅片相比，这样的系统更接近真实空间操作，有助于提前理解手术难点。这里还有一种“虚拟现实增强可微仿真”的思路，其含义可以简要理解为：把可微分的物理仿真器与虚拟现实平台结合起来，让系统不仅能展示操作过程，还能辅助轨迹规划与优化。这一技术也表明医疗图形学正在从“被动显示”走向“主动推演和辅助决策”。
2. 增强现实在手术中的作用，则更强调把虚拟信息叠加到真实视野中。手术规划路径、三维解剖结构和实时影像信息，不再只停留在屏幕上，而是可以在空间上与医生当前看到的手术区域对齐，从而形成更直观的导航提示。这里最关键的是 AR 必须在坐标系上正确配准，并且要让显示结果真正有助于定位和操作。否则虚拟信息一旦漂移或对齐不准，反而会带来误导。
3. 再进一步，图形学还被用于**个性化医疗与药物设计**。所谓个性化治疗，也常被称为精准医疗，其目标是根据患者的遗传信息、病史、生活习惯和影像数据，为不同个体设计更合适的方案。图形学在这里的角色，往往体现为**虚拟患者建模**。通过 CT、MRI 等医学影像重建出患者专属的三维解剖结构，再结合其他生物学信息，就有可能建立更完整的患者数字模型。这样一来，疾病分析、治疗规划甚至部分药物反应模拟，都可以在“虚拟患者”层面提前进

行。可以看出，在医疗健康领域，图形学的价值核心始终是帮助人更准确地理解真实结构和真实过程。

接下来，我们把视线转向另一类应用，也就是主观真实的重要代表：**CAD 与工业设计**。这里的“真实”更多体现为设计是否合理、表达是否清楚、模型是否可修改以及是否能服务后续制造。在第一章我们就接触过 CAD，也就是计算机辅助设计。它指的是利用计算机及图形设备帮助人完成设计工作的技术体系，其发展在图形学史上也具有非常重要的地位。二十世纪六十年代，MIT 推动交互式图形学研究，Ivan Sutherland 开发的 **Sketchpad** 通常被视为现代 CAD 的奠基性成果之一。它不只是一个画图程序，更重要的是展示了人与计算机通过图形方式直接交互的可能性。到了七十年代，随着小型计算机成本下降，交互式绘图系统开始走向工业应用；八十到九十年代，随着图形工作站普及和算法成熟，CAD 软件进一步进入机械、建筑、航空航天等多个工业领域，AutoCAD、CATIA、SolidWorks 等系统也相继成为主流工具。

CAD 的意义不应只理解为“在电脑上画图”。它真正改变的是设计与生产的整体流程。现代工业设计往往不只包含 CAD，还会与 **CAM** (Computer-Aided Manufacturing, 计算机辅助制造) 和 **CAE** (Computer-Aided Engineering, 计算机辅助工程) 协同工作。CAD 提供几何模型与设计表达，CAE 做分析和仿真，CAM 则把设计进一步连接到制造环节。于是，设计不再停留在图纸层面，而是形成可模拟、可验证、可制造的数字流程。

在 CAD 的核心技术中，最基础的是**几何建模方法**。两个最典型的代表是 **CSG** (Constructive Solid Geometry, 构造实体几何) 和 **B-Rep** (Boundary Representation, 边界表示)。CSG 的思路比较直观：先从立方体、圆柱体、圆锥体等基本几何体出发，再通过并、交、差等布尔运算，把它们组合成更复杂的模型。它的优点是表达清晰、构造速度快，还能较完整地保留组成实体的原始特征参数，因此修改起来比较方便。但它也有明显局限，例如精细曲面表达能力有限，难以直接保存完整的顶点、边、面细节，而且布尔运算过程中可能出现裂缝或空洞等几何问题。

与之相比，B-Rep 更强调显式地存储模型的边界信息，也就是顶点、边和面。它能更精确地表示复杂曲面，也更利于生成线框图、投影图以及计算几何特性。在很多现代 CAD 系统中，B-Rep 是非常核心的表示方式。进一步地，CAD 还发展出了**参数化建模与特征建模**。参数化建模把几何形状与参数和约束联系起来，于是只要调整参数值，设计就能快速更新；特征建模则在参数化框架上进一步引入孔、槽、倒角等工程特征，使模型更贴近真实制造语境。可以看到，这条路线本质上是在不断增强模型的“可修改性”和“工程语义”。

近年来，工业设计又出现了一个非常醒目的趋势，那就是 CAD 与人工智能的结合。首先是**文本生成 CAD**。以 **Text2CAD** 为代表的方法，希望把自然语言描述直接翻译成参数化 CAD 构造序列。这里的关键不只是“生成一个像样的三维外形”，而是生成一个**可编辑、可重用、保留构造历史**的 CAD 模型。这样一来，用户输入“画两个同心圆并沿 Z 轴拉伸形成中空结构”之类的描述，系统输出的就不只是最终形状，还包括构造过程本身。这一点十分重要，因为在工业设计里，后续修改往往和最终结果同样重要。

与文本生成对应的，是**图片生成 CAD**。在这里，**Img2CAD** 代表的就是从单张图像恢复参数化 CAD 模型的路线。它的难点在于，普通照片中看到的是表面视觉结果，而 CAD 模型需要的是可编辑的结构化构造序列。为了完成这件事，系统往往会先借助视觉-语言模型理解图像中的部件语义，例如识别出“椅背”“椅腿”等，再去预测各部分对应的草图、拉伸、尺寸和角度。换句话说，它是在尝试把视觉内容还原为更适合设计修改的参数化表示。

除了文本和图片，还有**草图生成 CAD**。其中，**PHT-CAD** 采用一种高效混合参数化表示方法，把直线、圆和圆弧统一编码，再结合视觉编码器与语言解码器，把草图图像转成结构化参数描述。这说明 AI 进入工业设计之后，核心目标并不只是“自动画东西”，而是要把人类熟悉的自然表达形式，如文字、照片、草图，重新接到 CAD 这种高度结构化的设计表示上。

还有一个很有意思的方向：**CAD 程序解析**。我们知道，在 CAD 中，很多绘图、修改和导出操作往往是重复的。通过编写脚本，用户可以把要在命令行手动输入的几十上百个指令写在文本里，这就是 CAD 脚本。当我们运行这个脚本时，程序就会像自动播放一样，依次执行里面的命令。而将复杂的 CAD 几何数据文件（如.dwg, .dxf 或 B-Rep 模型）转化为计算机图形学系统可以识别、编辑和渲染的底层逻辑与数学表达的过程，就是 CAD 程序解析。

传统上，人们更多关注怎样生成 CAD 脚本；但如果系统能读懂已有 CAD 脚本，就能支持自动修改、设计迁移和语义分析。**CADTalk** 就体现了这种思路。它通过语法树等方式把程序切分为可分析的代码块，再结合三维形状、多视角图像、开放词汇检测和分割模型，把视觉语义反向映射回程序结构。这样，模型就不只是在“看图识物”，而是在尝试理解“哪个程序片段对应模型的哪个部分”。这对未来的智能编辑和自动优化具有很强的启发意义。

最后一大类应用，是**影视制作**。这一领域的目标与医疗健康截然不同。医疗强调客观准确，而影视更关心视觉表现、表演还原和观众感受到的可信度。图形学在影视中最显眼的应用就是数字特效、角色动画和动作捕捉。其核心任务之一，是让虚拟角色尽可能自然地继承真实演员的动作与表情，同时又保留后期制作的自由度。为什么越来越重视虚拟角色呢？比如，在拍摄超越人类物理极限、非人生物与角色时，真人演员常常无法做到预期的动作。在《阿凡达：水之道》等电影中，很多场景完全由计算机生成。为了让画面中的数字角色与实拍场景的光影、环境完美融合，并且表现出人类演员的表演深度，必须通过动作捕捉等测量手段，将演员的表演高精度地复制到虚拟骨架上。

其中，**Flux** 系统体现的是数字面部特效中的一类思路。它希望减少演员在表演时对专门头盔（装有朝向面部的微型摄像头，记录脸上的每一个细微表情）、追踪点（演员须穿戴动捕服，捕捉身体动作）和强制约束（支架、威亚等）的依赖，使表演更加自然，然后再通过多摄像机拍摄、网格生成、数字资产匹配、红外数据分析和图像校准等流程，把真实演员的表演转化为高质量的数字面部效果。这里的数字资产，就是特定的 3D 模型、材质、特效或环境资产。在开发中，我们需要将其按特定的规则或语义，准确地分配、映射或融合到虚拟场景中。对于影视制作而言，这样做的重要意义在于：既要保留表演的细腻性，又要避免数字角色面部常见的僵硬感和不协调感。

这里的**动作捕捉** (Motion Capture) 就是一个值得关注的重点。如果完全依赖手工建模与关键帧动画来制作复杂角色动作，工作量会非常巨大。动作捕捉的思路是先测量真实演员的运动，再把这些运动映射到虚拟角色身上。这样能显著提高效率，也能提升动作的自然程度。按原理主要可划分为六类：机械式、声学式、电磁式、光学式、惯性式和无标记动作捕捉。其中，我们主要了解前五类：

1. 机械式动作捕捉通过刚性连杆与角度传感器来测量关节变化，从而推算末端点的空间位置和轨迹。它的优点是成本较低、精度较高，而且可实时测量；缺点则是装置本身会限制表演者的自由动作，因此可捕捉动作类型有限。
2. 声学式动作捕捉依靠超声波发射器和接收器，通过时间差或相位差来计算位置。它成本较低，也可以覆盖全身动作，但延迟较大、实时性和精度相对较弱。
3. 电磁式动作捕捉则通过电磁发射器与接收器推算空间位置和方向，实时性好、技术成熟、成本也相对可控，但它非常怕金属环境干扰，而且对剧烈运动的适应性有限。
4. 光学式动作捕捉在影视行业中尤其常见。它的基本原理是：用两台或更多摄像机从不同角度同时拍摄空间中的同一个标记点，由此恢复该点的三维位置；若以足够高的速率连续拍摄，就能恢复其运动轨迹。光学式又可分为主动式和被动式。主动式标记点会主动发光，速度快、精度高，在复杂光照环境下也较稳定，但标记点需要供电，且容易被遮挡。被动式标记点则通常采用反光材料，通过外部光源照明后反射回摄像机，不需要电源，佩戴更轻便，也更适合大规模布点。现代大型影视制作中，常见的就是多机位高速摄像机配合被动反光标记点的系统。
5. 惯性式动作捕捉则使用惯性传感器记录身体各部位的加速度、方向和倾斜角等信息。它的一个重要优势是对外部基础设施依赖较小，不需要布设复杂摄像机系统，也不太受光照条件限制，因此具有很高的移动自由度。不过它容易产生累计误差和漂移，这意味着捕捉时间一长，虚拟角色动作可能会逐渐偏离真实动作。

除了身体动作之外，影视制作还越来越重视**面部表情迁移**。和肢体动作相比，面部表情的难点在于它涉及大量细微变化，不只是眉毛和嘴角的位置移动，还包括额部皱纹、眼球转动、瞳孔变化以及许多微小肌肉联动。人类面部约有 43 块表情肌，这一数字本身就说明了问题的复杂性。要把这些微小变化稳定地记录并迁移到数字角色上，需要专门的捕捉、建模与驱动技术体系。也正因为如此，面部表情捕捉与迁移常被看作影视数字角色真实性的关键环节之一。

综上，真实图形学应用展示了图形学如何在不同目标下发挥作用的图景。医疗健康强调客观结构的准确重建与可解释分

析；CAD 与工业设计强调参数化、可编辑性和与制造流程的衔接；影视制作则强调视觉可信度、自然表演与高效生产。它们虽然服务对象不同，但都说明了一件事：图形学从来不只是“生成好看的图像”，而是一整套帮助人理解、设计、模拟和表达复杂世界的技术体系。

本节小练习

1. 简答题

为什么说医疗健康中的图形学应用更偏向“客观真实”，而影视制作中的图形学应用更偏向“主观真实”？

2. 简答题

请分别说明 CSG 和 B-Rep 在 CAD 建模中的基本思想，并概括它们各自的主要优点。

3. 判断题

Text2CAD、Img2CAD 这类方法的目标，主要只是生成一个外形看起来相似的三维网格模型，而不强调后续编辑与构造历史。（____）

参考解答

1. 因为医疗健康中的许多应用直接服务于诊断、治疗和手术规划，系统必须尽量准确反映患者真实的解剖结构和空间关系，所以更强调客观真实。影视制作则更重视观众感受到的可信度、角色表演是否自然、画面效果是否成立，因此它追求的是一种在视觉和叙事上令人信服的主观真实。
2. CSG 的基本思想是从立方体、圆柱体、圆锥体等基本几何体出发，通过并、交、差等布尔运算组合出复杂模型。它的优点是表达直观、生成速度快，而且便于保留原始特征参数。B-Rep 的基本思想是显式存储模型的顶点、边和面等边界信息来描述形状。它的优点是能更精确地表示复杂曲面，也更有利于进行投影、显示和几何特性计算。
3. 错。这类方法的价值恰恰在于它们希望输出可编辑、可修改、可重用的参数化 CAD 构造序列，而不仅仅是一个外观看起来相似但难以继续编辑的最终网格。